N NETLAB · DOCUMENTAÇÃO TÉCNICA

TUTORIAL COMPLETO DE CONSTRUÇÃO

Como o NetLab foi feito

Um guia didático sobre a arquitetura, os modelos de dados, as rotas, as validações e as decisões técnicas por trás do sistema de gerenciamento de infraestrutura de redes.

STACK

PYTHON · FLASK
SQLALCHEMY · POSTGRESQL

CONTEÚDO

8 MODELS · 28 ROTAS
18 TEMPLATES

VERSÃO

1.0 · NOVEMBRO DE 2026
BASEADO NO CÓDIGO-FONTE

Sumário

01 **Visão geral do NetLab**
- O problema que o sistema resolve
- O que o sistema faz hoje
- Escopo, limites e números do banco

02 **Stack e decisões de base**
- Por que Flask, SQLAlchemy e PostgreSQL
- Renderização no servidor com Jinja2
- O que foi deliberadamente deixado de fora

03 **Estrutura do projeto**
- Papel de cada pasta e arquivo
- O caminho de uma requisição

04 **Como a aplicação inicia**
- Configuração, blueprints e banco
- Login e o before_request
- O que db.create_all() faz — e o que não faz

05 **Modelagem de dados**
- Os 8 models, campo por campo
- Relacionamentos e integridade
- As decisões de modelagem explicadas

06 **Rotas e blueprints**
- Mapa completo das 28 rotas
- Regiões, fabricantes, modelos e equipamentos
- Conexões, API JSON e autenticação

07 **Fluxos completos, passo a passo**
- Do cadastro da região ao cabo conectado
- Exclusão em cascata e regras de integridade

08 **Templates e interface**
- Jinja, flash messages e macros recursivas
- JavaScript da seleção de modelos
- O design system Termix

09 **Login, sessão e segurança**
- Como a autenticação funciona
- Limitações conscientes do modelo atual

10 **Como executar o projeto**
- Dependências, PostgreSQL e ambiente virtual
- Primeiro acesso e teste de conexão

11 **Guia rápido de uso**
- Receitas para cada tarefa do dia a dia

12 **Manutenção e diagnóstico**
- Como adicionar um novo CRUD
- Erros comuns e o que eles significam
- A lição do campo slug

13 **Dívidas técnicas e próximos passos**
- O que melhorar, com prioridade

14 **Glossário**
- Termos de Flask, banco e redes

15 **Apêndices**
- A — Mapa de arquivos
- B — Comandos úteis
- C — Checklist de verificação

> **COMO LER ESTE DOCUMENTO**
>
> Os capítulos 1 a 4 explicam o contexto e a fundação; 5 e 6 são a referência de dados e rotas; 7 mostra o sistema em movimento; 8 a 12 cobrem interface, segurança, execução e manutenção; 13 lista as limitações atuais. Quem só quer rodar o projeto pode ir direto ao capítulo 10.

01 Visão geral do NetLab

O NetLab é uma aplicação web de gerenciamento de infraestrutura de redes. Ele foi pensado para responder a uma pergunta simples, mas que fica difícil de responder quando a rede cresce: **o que existe, onde está e como está conectado?** Para isso, o sistema organiza a infraestrutura em cinco camadas: locais (regiões), fabricantes, modelos de equipamento, equipamentos reais e conexões físicas entre suas portas.

1.1 O problema que o sistema resolve

Em uma planilha, é comum guardar uma lista de equipamentos com nome, modelo e sala. O problema aparece quando é preciso saber, por exemplo, quais portas do switch do laboratório estão livres, qual é o modelo exato daquele roteador, ou se um computador está ligado na porta correta. Uma planilha não impede que duas pessoas cadastrem o mesmo modelo com nomes diferentes, não valida que a porta de destino existe e não mantém o histórico de conexões consistente.

O NetLab troca a planilha por um banco relacional com validações na aplicação. Cada conceito vira uma tabela, cada ligação vira uma chave estrangeira e cada operação perigosa passa por uma checagem antes de chegar ao banco. O resultado é um inventário navegável: da região até a porta, e da porta até o cabo.

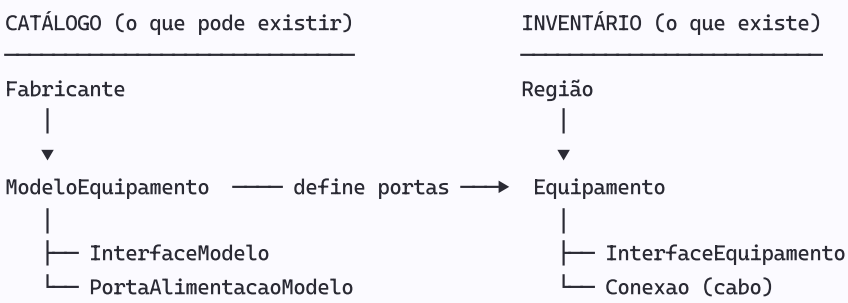
1.2 O que o sistema faz hoje

MÓDULO	O QUE PERMITE FAZER
Regiões	Criar locais e sublocais em hierarquia (campus → prédio → sala), editar, filtrar por subárvore e excluir com proteções.
Fabricantes	Manter o catálogo de marcas, com nome único, edição e exclusão bloqueada quando há modelos vinculados.
Modelos	Cadastrar modelos com altura em U, peso, fluxo de ar e profundidade, além de pesquisar, filtrar por fabricante e paginar.
Interfaces do modelo	Descrever as portas que um modelo possui (nome, tipo, se está habilitada e se é de gerenciamento), com paginação.
Portas de alimentação	Registrar as entradas de energia previstas no modelo, com tipo e descrição.
Equipamentos	Instanciar um modelo em uma região, gerando automaticamente as portas físicas do equipamento a partir do modelo.
Portas do equipamento	Ver o estado de cada porta (disponível, conectada ou desabilitada) e iniciar ou encerrar uma conexão.
Conexões	Ligar duas portas de equipamentos diferentes, com tipo de cabo e descrição, impedindo que uma porta seja usada duas vezes.
Login	Exigir usuário e senha para acessar qualquer página interna, com sessão e logout.

1.3 Catálogo versus inventário

A distinção mais importante do domínio é entre **catálogo** e **inventário**. O catálogo descreve o que *pode* existir: o fabricante "Cisco", o modelo "Catalyst 2960" e as 24 portas que esse modelo possui. O inventário descreve o que *existe de fato*: o switch chamado "SW-LAB-01", instalado na sala 204, com 24 portas físicas, das quais duas estão conectadas.

Figura 1.1 — Separação entre catálogo e inventário



Essa separação é o que permite copiar as interfaces do modelo no momento em que o equipamento é cadastrado. O modelo funciona como uma "planta" e o equipamento como uma "unidade construída" a partir dela. Alterações posteriores no catálogo não reescrevem o passado do inventário — o que é uma decisão deliberada, explicada no capítulo 5.

1.4 Escopo, limites e números do banco

O sistema é uma aplicação **local**, executada com o servidor de desenvolvimento do Flask, e foi construído com foco em clareza e aprendizado. Não há API pública, aplicativo móvel, controle de usuários por perfil, auditoria ou integração com equipamentos reais via SNMP. O login é único e fixo, e o banco roda no próprio computador.

A base atual, usada para validar o projeto, tem os seguintes volumes:

TABELA	REGISTROS	LEITURA
`regioes`	2	Poucas regiões, com hierarquia.
`fabricantes`	374	Catálogo amplo de marcas.
`modelos_equipamento`	12.069	Catálogo grande; justifica paginação.
`interfaces_modelo`	117.605	Muitas portas por modelo; justifica paginação de 50.
`portas_alimentacao_modelo`	9.378	Entradas de energia do catálogo.
`equipamentos`	1	Inventário ainda em formação.
`interfaces_equipamento`	1	Portas físicas copiadas do modelo.
`conexoes`	0	Nenhum cabo registrado até o momento.

POR QUE OS NÚMEROS IMPORTAM

Um catálogo com mais de 12 mil modelos e 117 mil interfaces muda decisões de projeto. Uma listagem simples de todas as interfaces em uma única página ficaria pesada para o navegador e para o banco. Por isso a rota de interfaces pagina de 50 em 50, e a de modelos, de 25 em 25. A paginação não é enfeite: é uma consequência direta do volume real de dados.

02 Stack e decisões de base

Toda escolha de tecnologia esconde uma troca. O NetLab priorizou **simplicidade operacional**, **explicitude** e **uma única linguagem** para a maior parte do trabalho. As versões abaixo são as que estavam instaladas no ambiente em que este documento foi gerado.

COMPONENTE	VERSÃO	PAPEL NO PROJETO
Python	3.14.6	Linguagem da aplicação.
Flask	3.1.3	Microframework web: rotas, sessão, templates, blueprints.
Flask-SQLAlchemy	3.1.1	Integra o SQLAlchemy ao ciclo de vida do app Flask.
SQLAlchemy	2.1.4	ORM: classes Python viram tabelas e consultas.
psycopg	3.3.6	Driver de conexão com o PostgreSQL.
Jinja2	3.1.6	Renderização dos templates HTML no servidor.
Werkzeug	3.1.9	Base do Flask: requisição, resposta, servidor de desenvolvimento.
PostgreSQL	—	Banco relacional local, base `netlab`.

2.1 Por que Flask

Flask foi escolhido por ser um microframework: ele entrega o essencial (roteamento, requisição, resposta, templates e sessão) e deixa o restante para bibliotecas separadas. Isso combina com o objetivo do projeto, que é entender cada peça. Em um framework mais completo, como Django, muita coisa viria pronta — admin, autenticação, migrações — e o aprendizado sobre "o que acontece por baixo" ficaria escondido.

O custo dessa escolha é que várias decisões precisam ser tomadas manualmente: organizar rotas em blueprints, validar formulários, tratar erros, proteger exclusões e criar a tela de login. O NetLab faz isso de forma explícita, e é justamente esse código explícito que este tutorial documenta.

2.2 Por que Flask-SQLAlchemy + SQLAlchemy

O SQLAlchemy é o ORM (Object-Relational Mapper): ele permite escrever `Regiao.query.order_by(Regiao.nome).all()` em vez de montar SQL à mão. Mais do que conveniência, o ORM oferece relacionamentos, carregamento sob demanda, transações e proteção contra vários tipos de erro de montagem de consulta.

O Flask-SQLAlchemy entra como adaptador. Ele cria o objeto `db`, registra os models e gerencia a sessão de banco dentro do contexto da aplicação Flask. É por isso que o projeto tem um `config.py` mínimo:

```
from flask_sqlalchemy import SQLAlchemy

# instanciando o objeto de conexao com o banco de dados
db = SQLAlchemy()
```

Esse objeto é importado pelos models e pelas rotas. O vínculo com o app só acontece em `app.py`, com `db.init_app(app)`. Esse padrão evita import circular: os models não precisam importar o app, apenas o objeto `db`.

2.3 Por que PostgreSQL

O banco escolhido foi o PostgreSQL, acessado pelo driver `psycopg` 3. As razões práticas são:

- **Integridade referencial real.** Chaves estrangeiras são verificadas pelo banco, não só pela aplicação.
- **Tipos adequados ao domínio.** `JSON` para guardar o payload original de um modelo importado, `BOOLEAN` para flags, `NUMERIC` / `FLOAT` para medidas.
- **Comportamento próximo de produção.** É o mesmo banco usado em muitos ambientes reais, o que torna o aprendizado transferível.
- **Robustez com volume.** O catálogo atual passa de 117 mil interfaces sem dificuldade.

A conexão está declarada diretamente em `app.py`:

```
app.config['SQLALCHEMY_DATABASE_URI'] = (
    'postgresql://postgres:4548@localhost:5432/netlab'
)
```

A URL segue o formato `postgresql://usuario:senha@host:porta/banco`. Neste caso: usuário `postgres`, senha `4548`, host local, porta padrão 5432 e banco `netlab`.

> **CREDENCIAL EM CÓDIGO**
>
> Deixar usuário e senha no código é aceitável para um projeto local de estudo, mas não para algo publicado. O capítulo 9 mostra como mover esses valores para variáveis de ambiente sem mudar a lógica do sistema.

2.4 Por que renderização no servidor

Os templates são renderizados pelo Jinja2 no servidor e enviados ao navegador como HTML pronto. Não há React, Vue ou etapa de build. Essa escolha reduz o número de ferramentas necessárias e mantém o fluxo fácil de acompanhar: o navegador pede uma URL, o Flask consulta o banco, o Jinja monta a página e o navegador exibe.

Ainda assim, há JavaScript onde ele agrega valor imediato: carregar os modelos de um fabricante sem recarregar a página e mostrar ou ocultar o filtro de regiões. É o mínimo necessário, sem transformar o projeto em uma SPA.

2.5 Por que CSS puro com variáveis

O visual é construído em um único `static/style.css`, com variáveis CSS no `:root`. Isso permite trocar a identidade visual inteira mudando um bloco de tokens, sem tocar nos templates. O tema atual, chamado Termix, usa fundo quase preto, fonte monoespaçada, bordas finas e acento roxo. O capítulo 8 detalha o design system.

2.6 O que foi deliberadamente deixado de fora

NÃO USADO	MOTIVO
Django	Traria admin e autenticação prontos, escondendo o funcionamento interno que o projeto quer demonstrar.
FastAPI	É excelente para APIs, mas o NetLab é majoritariamente renderização de páginas com formulários.
React / Vue	Exigiriam build, estado no cliente e uma camada extra de complexidade sem ganho para o escopo.
SQLite	Simplificaria a instalação, mas não exercitaria um banco cliente-servidor com os tipos e a concorrência do PostgreSQL.
Flask-Migrate / Alembic	Não foram adotados; por isso alterações de schema são manuais. É uma dívida técnica consciente (capítulo 12).
Flask-WTF / CSRF	Os formulários usam HTML puro. A proteção CSRF fica como melhoria futura.

03 Estrutura do projeto

O projeto segue uma organização por responsabilidade: a criação do app fica na raiz, a conexão com o banco em `config.py`, as tabelas em `models/`, as URLs em `rotas/`, o HTML em `templates/` e o estilo em `static/`.

Figura 3.1 — Árvore de arquivos relevantes

```
Projeto NetLab/
├── app.py                  # cria o Flask, configura, registra blueprints, login e rotas base
├── config.py               # instancia o SQLAlchemy (db)
├── requerimentos.txt       # dependências (arquivo em UTF-16)
├── GUIA_CRUD.md            # guia de referência dos CRUDs
├── README.md               # descrição inicial do projeto
│
├── models/                 # 8 classes que representam tabelas
│   ├── regiao.py
│   ├── fabricante.py
│   ├── modelo_equipamento.py
│   ├── interface_modelo.py
│   ├── porta_alimentacao_modelo.py
│   ├── equipamento.py
│   ├── interface_equipamento.py
│   └── conexao.py
│
├── rotas/                  # funções que respondem às URLs
│   ├── regioes.py          # CRUD de regiões + hierarquia
│   ├── fabricantes.py      # CRUD de fabricantes
│   ├── modelos.py          # modelos, interfaces e portas de alimentação
│   ├── equipamentos.py     # equipamentos, portas, API de modelos e exclusão em cascata
│   ├── conexoes.py         # criação, listagem e remoção de cabos
│   └── sistema.py          # legado: não está registrado no app.py
│
├── templates/              # 18 templates Jinja
│   ├── index.html
│   ├── login.html
│   ├── regiao.html / editar_regiao.html
│   ├── fabricantes.html / editar_fabricante.html
│   ├── modelos.html / editar_modelo.html
│   ├── interfaces_modelo.html / editar_interface_modelo.html
│   ├── portas_alimentacao_modelo.html / editar_porta_alimentacao_modelo.html
│   ├── equipamentos.html / detalhe_equipamento.html / editar_equipamento.html
│   ├── portas_equipamento.html / nova_conexao.html / conexoes.html
│   └── ...
│
└── static/
    └── style.css           # tema Termix, layout e componentes (~930 linhas)
```

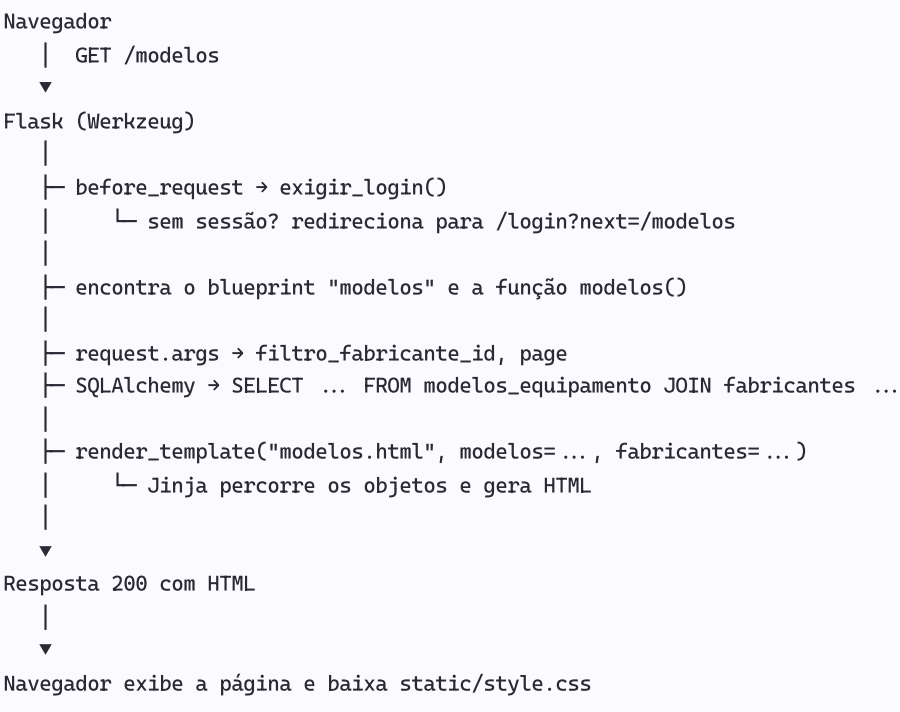
3.1 O papel de cada pasta

CAMINHO	RESPONSABILIDADE
`app.py`	Ponto de entrada. Cria o objeto Flask, define configurações, inicializa o banco, registra os cinco blueprints, implementa login/logout, `/` e `/teste-banco`, e chama `db.create_all()`.
`config.py`	Única responsabilidade: criar `db = SQLAlchemy()`. Separar isso evita import circular entre app, rotas e models.
`models/`	Cada arquivo declara uma classe `db.Model` com `__tablename__`, colunas e relacionamentos. Model não é CRUD: é apenas a descrição da tabela.
`rotas/`	Cada arquivo declara um `Blueprint` e as funções que leem `request`, validam, consultam o banco, gravam e renderizam ou redirecionam.
`templates/`	HTML com sintaxe Jinja. Recebem variáveis do `render_template` e montam tabelas, formulários e mensagens.
`static/`	Arquivos servidos diretamente ao navegador. Hoje contém apenas o CSS.

3.2 O caminho de uma requisição

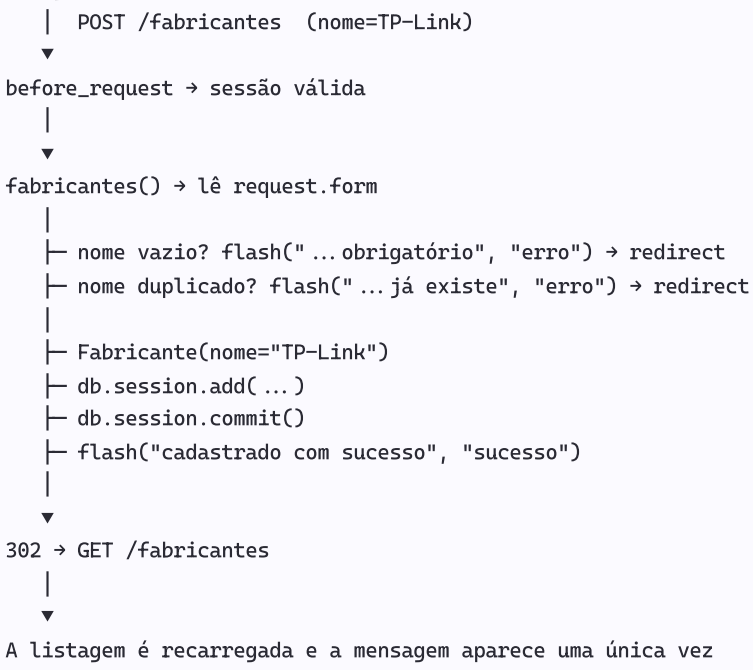
Entender o percurso de uma requisição é entender o projeto inteiro. Quando o usuário abre `/modelos`, acontece o seguinte:

Figura 3.2 — Ciclo de uma requisição GET



Em uma operação de escrita, o caminho muda apenas no meio: a função lê `request.form`, valida os dados, cria ou altera objetos, chama `db.session.commit()`, registra uma mensagem com `flash()` e responde com `redirect()`. O padrão **POST → redirect → GET** evita que o usuário reenvie o formulário ao atualizar a página.

Figura 3.3 — Ciclo de uma requisição POST



Esse desenho é intencional: a validação acontece no servidor, a mensagem sobrevive a um redirecionamento e o navegador nunca fica com um estado "meio enviado".

04 Como a aplicação inicia

O `app.py` tem 104 linhas e concentra a fundação do sistema. Ele pode ser lido em quatro blocos: criação e configuração, registro dos blueprints, autenticação e rotas base, e inicialização do banco.

4.1 Criação e configuração

```
from flask import *
from config import db

from rotas.conexoes import bp_conexoes
from rotas.regioes import bp_regioes
from rotas.equipamentos import bp_equipamentos
from rotas.fabricantes import bp_fabricantes
from rotas.modelos import bp_modelos

app = Flask(__name__)

app.config['SECRET_KEY'] = 'chave-temporaria-netlab'
app.config['SQLALCHEMY_DATABASE_URI'] = (
    'postgresql://postgres:4548@localhost:5432/netlab'
)
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db.init_app(app)
```

CONFIGURAÇÃO	PARA QUE SERVE
`SECRET_KEY`	Chave usada para assinar o cookie de sessão. Sem ela, o Flask não consegue manter o usuário logado com segurança mínima. O valor atual é provisório e deveria vir de variável de ambiente.
`SQLALCHEMY_DATABASE_URI`	Endereço do banco PostgreSQL. É a única linha que precisa mudar para apontar para outro servidor ou outra base.
`SQLALCHEMY_TRACK_MODIFICATIONS`	Desligado para evitar o rastreamento de modificações do SQLAlchemy, que consome memória e não é usado pelo projeto.

O uso de `from flask import *` funciona porque o Flask exporta os nomes necessários (`Flask`, `request`, `render_template`, `redirect`, `url_for`, `flash`, `session`, `current_app`). É prático em um projeto pequeno, mas dificulta saber de onde cada nome veio. Em uma refatoração futura, o ideal é importar explicitamente, como já é feito em `rotas/fabricantes.py` e `rotas/modelos.py`.

4.2 Registro dos blueprints

```
app.register_blueprint(bp_regioes)
app.register_blueprint(bp_equipamentos)
app.register_blueprint(bp_fabricantes)
app.register_blueprint(bp_modelos)
app.register_blueprint(bp_conexoes)
```

Um **blueprint** é um conjunto de rotas agrupado por assunto, sem uma aplicação própria. Ele permite dividir o sistema em arquivos sem perder a capacidade de usar `url_for` com nome qualificado, como `url_for("modelos.modelos")`. O primeiro trecho é o nome do blueprint; o segundo, o nome da função.

Note que `rotas/sistema.py` existe, tem duas rotas (`/` e `/teste-banco`), mas **não está registrado**. As mesmas rotas foram implementadas diretamente em `app.py`. O arquivo é um resquício de uma organização anterior e pode ser removido ou reaproveitado; hoje ele não afeta a aplicação.

4.3 Login e rotas base

```
USUARIO_PADRAO = "admin"
SENHA_PADRAO = "netlab"


@app.before_request
def exigir_login():
    # Libera a própria tela de login e os arquivos estáticos.
    if request.endpoint in ("login", "static"):
        return None

    if not session.get("usuario"):
        return redirect(url_for("login", next=request.path))
```

`before_request` é um gancho executado antes de qualquer rota. Ele funciona como uma portaria: se não há `usuario` na sessão, a requisição é desviada para `/login`, levando no parâmetro `next` o caminho original. As exceções são a própria rota de login e os arquivos estáticos, que precisam ser acessíveis para a tela de login carregar o CSS.

A rota `/login` aceita GET e POST. No GET, apenas renderiza o formulário. No POST, compara usuário e senha com as constantes. Se conferem, grava `session["usuario"]` e redireciona para o destino seguro. Se não, devolve a mesma página com a mensagem “Usuário ou senha inválidos.”.

4.4 Inicialização do banco

```
with app.app_context():
    db.create_all()


if __name__ == "__main__":
    app.run(debug=True)
```

`db.create_all()` percorre os models que foram importados e cria as tabelas que ainda não existem. Ele é executado dentro de um **contexto de aplicação**, porque o SQLAlchemy precisa saber a qual app e a qual banco a operação pertence.

> **O QUE DB.CREATE_ALL() NÃO FAZ**
>
> Ele **não altera tabelas existentes**. Se uma coluna for adicionada, removida ou tiver o tipo alterado no model, o banco continuará com o schema antigo. Nesse caso é preciso rodar um `ALTER TABLE` manual ou adotar uma ferramenta de migração. Esse comportamento já causou um problema real no projeto, envolvendo o antigo campo `slug` (capítulo 12).

Outro detalhe importante: `create_all()` só enxerga os models que foram carregados na memória. Como `app.py` importa os blueprints, e cada blueprint importa seus models, todos os oito models acabam registrados antes da chamada. Se um model novo não for importado em lugar nenhum, a tabela dele não será criada — e o erro só aparecerá na primeira consulta.

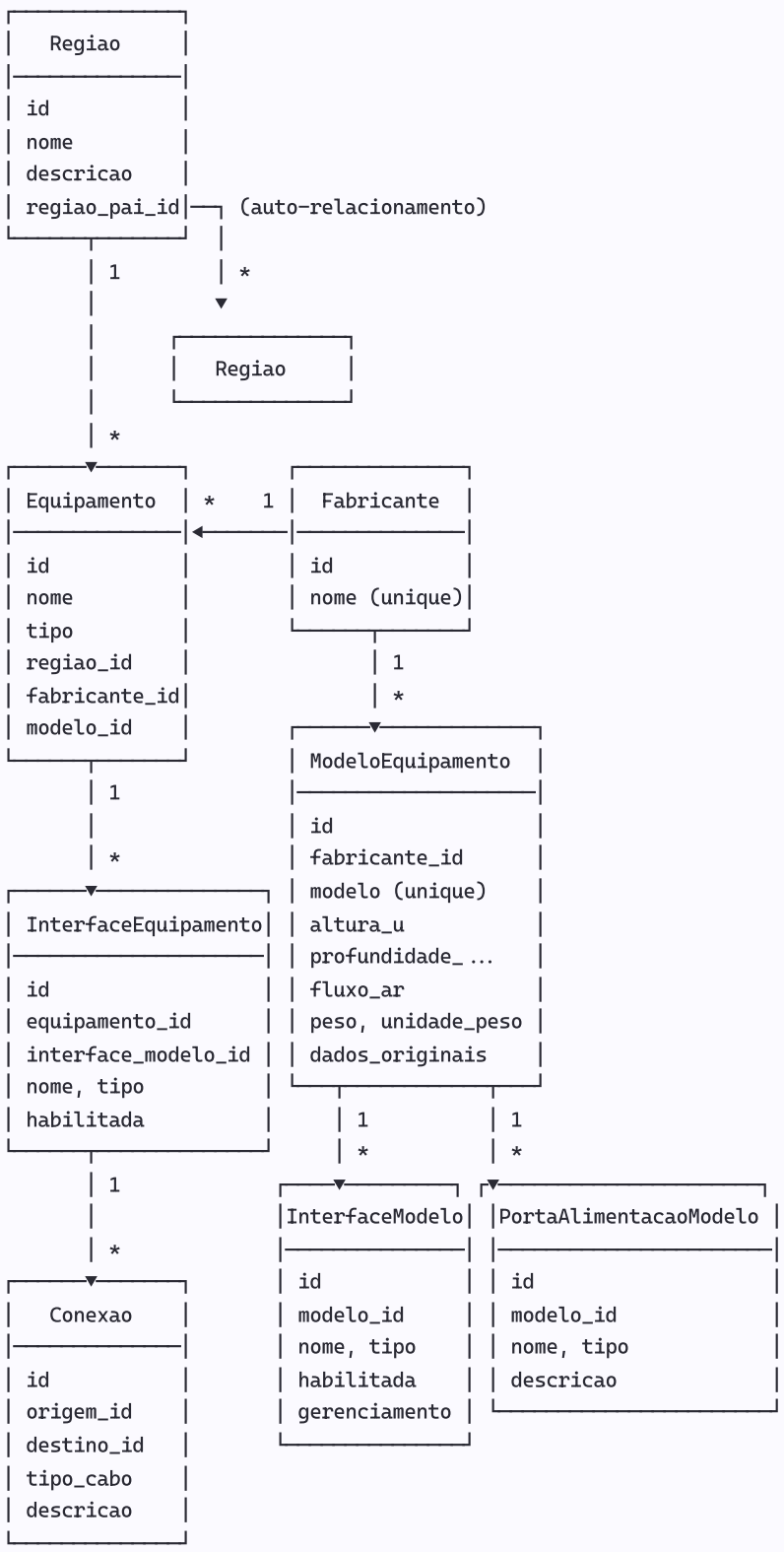
Por fim, `app.run(debug=True)` sobe o servidor de desenvolvimento com recarregamento automático e páginas de erro detalhadas. É ótimo para desenvolvimento e inadequado para produção: o debugger interativo nunca deve ficar exposto.

05 Modelagem de dados

O banco tem oito tabelas. A modelagem segue uma direção clara: o catálogo (fabricante → modelo → interfaces/portas) alimenta o inventário (região → equipamento → interfaces → conexões). As chaves estrangeiras amarram essa cadeia, e os relacionamentos do SQLAlchemy permitem navegar nos dois sentidos.

5.1 Visão geral do domínio

Figura 5.1 — Relacionamentos entre as tabelas



Leia o diagrama em duas metades. A metade superior é o catálogo: um fabricante tem muitos modelos; um modelo tem muitas interfaces e muitas portas de alimentação. A metade inferior é o inventário: uma região

tem muitos equipamentos; um equipamento tem muitas interfaces; e uma conexão liga duas interfaces de equipamentos.

5.2 Regiao

Representa um local: campus, prédio, andar, sala, setor ou qualquer outra divisão que faça sentido. A hierarquia é feita com um **auto-relacionamento**: cada região pode apontar para outra região como sua "mãe".

COLUNA	TIPO	REGRA
`id`	Integer	Chave primária.
`nome`	String(150)	Obrigatório.
`descricao`	String(1000)	Opcional.
`regiao_pai_id`	Integer / FK	Opcional; aponta para `regioes.id`. Nulo = região raiz.

```
regiao_pai = db.relationship(
    "Regiao",
    remote_side=[id],
    back_populates="subregioes"
)

subregioes = db.relationship(
    "Regiao",
    back_populates="regiao_pai"
)
```

`remote_side=[id]` diz ao SQLAlchemy qual lado do relacionamento é o "um". Sem isso, o ORM não saberia distinguir pai de filho em uma tabela que aponta para si mesma. Com `back_populates`, as duas pontas ficam sincronizadas: atribuir `filha.regiao_pai = pai` também coloca a filha em `pai.subregioes`.

A hierarquia é **opcional** e sem limite de profundidade. Uma região raiz tem `regiao_pai_id = None`; suas filhas apontam para ela, e assim por diante. A aplicação monta uma lista achatada com o nível de cada item para exibir o seletor com recuo, e usa recursão para mostrar a árvore na página. O capítulo 6 explica a prevenção de ciclos.

5.3 Fabricante

COLUNA	TIPO	REGRA
`id`	Integer	Chave primária.
`nome`	String(100)	Obrigatório e **único**.

```
class Fabricante(db.Model):
    __tablename__ = 'fabricantes'

    id = db.Column(db.Integer, primary_key=True)
    nome = db.Column(db.String(100), nullable=False, unique=True)

    modelos = db.relationship("ModeloEquipamento", back_populates="fabricante")
```

A unicidade do nome é garantida em dois níveis: a rota verifica antes de inserir e o banco também rejeita duplicatas por meio da constraint `unique`. Isso protege contra duas requisições simultâneas que passariam pela checagem da aplicação ao mesmo tempo.

5.4 ModeloEquipamento

É a tabela mais rica do catálogo. Descreve um modelo comercial: o nome, o fabricante, as dimensões físicas e metadados de importação.

COLUNA	TIPO	REGRA E SIGNIFICADO
`id`	Integer	Chave primária.
`fabricante_id`	Integer / FK	Obrigatório; aponta para `fabricantes.id`.
`modelo`	String(150)	Obrigatório e único no banco.
`altura_u`	Float	Obrigatório, padrão 0.0. Altura em unidades de rack (U).
`profundidade_completa`	Boolean	Obrigatório, padrão `False`. Indica se o equipamento ocupa a profundidade inteira do rack.
`fluxo_ar`	String(50)	Opcional. Direção do fluxo de ar (frontal, traseiro, lateral etc.).
`peso`	Float	Opcional. Peso numérico.
`unidade_peso`	String(20)	Opcional. Unidade do peso (kg, lb etc.).
`dados_originais`	JSON	Opcional. Payload bruto de importação, preservado sem virar colunas.

```
interfaces = db.relationship(
    "InterfaceModelo",
    back_populates="modelo",
    cascade="all, delete-orphan"
)

portas_alimentacao = db.relationship(
    "PortaAlimentacaoModelo",
    back_populates="modelo",
    cascade="all, delete-orphan"
)
```

O `cascade="all, delete-orphan"` significa que, ao excluir um modelo, suas interfaces e portas de alimentação também são excluídas. "Orphan" cobre o caso de um filho que deixa de pertencer ao pai: ele é removido em vez de ficar órfão no banco. Isso faz sentido porque uma interface de modelo não existe sem o modelo.

> **UMA DECISÃO QUE MERECE REVISÃO**
>
> `modelo` é único globalmente. Isso impede que dois fabricantes diferentes tenham um modelo com o mesmo nome — situação plausível no mundo real (por exemplo, nomes genéricos repetidos entre marcas). O mais correto seria uma constraint única composta por `(fabricante_id, modelo)`. A regra atual simplifica o catálogo e evita ambiguidade na busca, mas é uma limitação conhecida.

5.5 InterfaceModelo

Descreve uma porta prevista no modelo: "GigabitEthernet0/1", "Console", "SFP1". É a planta baixa que será copiada para o equipamento no momento do cadastro.

COLUNA	TIPO	REGRA E SIGNIFICADO
`id`	Integer	Chave primária.
`modelo_id`	Integer / FK	Obrigatório; aponta para `modelos_equipamento.id`.
`nome`	String(100)	Obrigatório. Ex.: `Gi0/1`.
`tipo`	String(100)	Obrigatório. Ex.: `Ethernet`, `Fibra`, `Console`.
`habilitada`	Boolean	Obrigatório, padrão `True`. Se falso, não é copiada para novos equipamentos.
`gerenciamento`	Boolean	Obrigatório, padrão `False`. Marca portas de gerência.

Não há constraint de unicidade no banco para `(modelo_id, nome)`. A unicidade é validada na rota: antes de inserir, o sistema consulta se já existe uma interface com aquele nome naquele modelo. A validação funciona, mas fica vulnerável a concorrência; uma constraint composta no banco seria mais robusta.

5.6 PortaAlimentacaoModelo

Registra as entradas de energia do modelo. É uma tabela simples, separada das interfaces de rede porque a natureza do dado é outra: uma porta de alimentação não recebe cabo de rede nem participa de conexões.

COLUNA	TIPO	REGRA
`id`	Integer	Chave primária.
`modelo_id`	Integer / FK	Obrigatório; aponta para `modelos_equipamento.id`.

COLUNA	TIPO	REGRA
`nome`	String(100)	Obrigatório. Ex.: `PSU 1`.
`tipo`	String(100)	Obrigatório. Ex.: `AC`, `DC`.
`descricao`	String(500)	Opcional.

A unicidade do nome também é validada na rota, por modelo. Como não há relacionamento com o inventário, excluir uma porta de alimentação do catálogo não afeta equipamentos já cadastrados.

5.7 Equipamento

É a instância real de um modelo em uma região. O equipamento guarda o nome escolhido pela equipe ("SW-LAB-01"), o tipo e as referências para região, fabricante e modelo. Fabricante e modelo são opcionais para permitir cadastrar dispositivos ainda sem catálogo definido.

COLUNA	TIPO	REGRA
`id`	Integer	Chave primária.
`nome`	String(150)	Obrigatório.
`tipo`	String(50)	Obrigatório. Roteador, Switch, Computador, Notebook, Celular, Servidor, Access Point ou Outro.
`regiao_id`	Integer / FK	Obrigatório; aponta para `regioes.id`.
`fabricante_id`	Integer / FK	Opcional; aponta para `fabricantes.id`.
`modelo_id`	Integer / FK	Opcional; aponta para `modelos_equipamento.id`.

```
regiao = db.relationship("Regiao")
fabricante = db.relationship("Fabricante")
modelo = db.relationship("ModeloEquipamento")
```

Repare que `Equipamento` **não declara** um relacionamento com `InterfaceEquipamento`. As portas são consultadas explicitamente por `equipamento_id` nas rotas. Isso deixa a exclusão em cascata manual, mas também torna o fluxo de exclusão totalmente visível: primeiro conexões, depois interfaces, por fim o equipamento.

5.8 InterfaceEquipamento

É a porta física de um equipamento real. Nasce como cópia de uma `InterfaceModelo`, mas guarda seus próprios valores de nome, tipo e estado, para que o inventário não mude se o catálogo mudar.

COLUNA	TIPO	REGRA E SIGNIFICADO
`id`	Integer	Chave primária.
`equipamento_id`	Integer / FK	Obrigatório; aponta para `equipamentos.id`.
`interface_modelo_id`	Integer / FK	Opcional; aponta para `interfaces_modelo.id`. Preserva a origem da cópia.
`nome`	String(100)	Obrigatório. Copiado do modelo.
`tipo`	String(100)	Obrigatório. Copiado do modelo.
`habilitada`	Boolean	Obrigatório, padrão `True`. Portas desabilitadas não podem ser conectadas.

O campo `interface_modelo_id` é opcional por dois motivos: permite portas criadas manualmente, sem origem no catálogo, e evita que a exclusão de uma interface de modelo quebre o inventário. Mesmo assim, a rota de exclusão de interface de modelo bloqueia a operação se houver equipamentos usando aquela interface — uma proteção a mais.

5.9 Conexao

Representa um cabo entre duas portas físicas. É uma tabela com duas chaves estrangeiras para a mesma tabela (`interfaces_equipamento`), o que exige dizer ao SQLAlchemy qual coluna cada relacionamento usa.

COLUNA	TIPO	REGRA
`id`	Integer	Chave primária.
`interface_origem_id`	Integer / FK	Obrigatório, **único**; aponta para `interfaces_equipamento.id`.
`interface_destino_id`	Integer / FK	Obrigatório, **único**; aponta para `interfaces_equipamento.id`.
`tipo_cabo`	String(50)	Obrigatório, padrão `Ethernet`.
`descricao`	String(250)	Opcional.

```
interface_origem = db.relationship(
    "InterfaceEquipamento",
    foreign_keys=[interface_origem_id]
)

interface_destino = db.relationship(
    "InterfaceEquipamento",
    foreign_keys=[interface_destino_id]
)
```

O `unique=True` nas duas pontas é a garantia estrutural de que uma porta participa de no máximo uma conexão. A aplicação também verifica isso antes de inserir, para produzir uma mensagem amigável em vez de um erro de banco. A dupla proteção é proposital: a checagem na rota melhora a experiência; a constraint no banco protege a integridade.

> **POR QUE NÃO EXISTE UMA TABELA "CABO" SEPARADA?**
>
> Porque o cabo, neste escopo, é a própria conexão ponto a ponto. Ele tem duas pontas, um tipo e uma descrição. Uma tabela separada só faria sentido se o mesmo cabo pudesse conectar mais de duas portas ou tivesse identidade física própria (patrimônio, comprimento, data de instalação), o que ainda não é o caso.

5.10 Resumo das tabelas

MODEL	TABELA	PK	CHAVES ESTRANGEIRAS	REGRAS PRINCIPAIS
`Regiao`	`regioes`	`id`	`regiao_pai_id` → `regioes.id`	Auto-relacionamento; exclusão bloqueada com filhas ou equipamentos.
`Fabricante`	`fabricantes`	`id`	—	`nome` único; exclusão bloqueada com modelos.
`ModeloEquipamento`	`modelos_equipamento`	`id`	`fabricante_id` → `fabricantes.id`	`modelo` único; cascata para interfaces e portas de alimentação.
`InterfaceModelo`	`interfaces_modelo`	`id`	`modelo_id` → `modelos_equipamento.id`	Nome único por modelo (validado na rota); exclusão bloqueada se usada.
`PortaAlimentacaoModelo`	`portas_alimentacao_modelo`	`id`	`modelo_id` → `modelos_equipamento.id`	Nome único por modelo (validado na rota).
`Equipamento`	`equipamentos`	`id`	`regiao_id`, `fabricante_id`, `modelo_id`	Região obrigatória; fabricante e modelo opcionais; modelo precisa pertencer ao fabricante.
`InterfaceEquipamento`	`interfaces_equipamento`	`id`	`equipamento_id`, `interface_modelo_id`	Cópia do modelo; portas desabilitadas não conectam.
`Conexao`	`conexoes`	`id`	`interface_origem_id`, `interface_destino_id`	Uma porta só participa de uma conexão; equipamentos diferentes.

5.11 As decisões de modelagem explicadas

Por que copiar as interfaces do modelo para o equipamento?

Porque o inventário precisa ser um **retrato do momento da instalação**. Se o catálogo do modelo for corrigido depois — uma porta renomeada, uma interface removida —, o equipamento já instalado não deve

mudar sozinho. A cópia também permite que o equipamento tenha estados próprios (habilitada/desabilitada) sem alterar o catálogo, que é compartilhado por todos os equipamentos daquele modelo.

O custo é a duplicação de dados e a necessidade de manter o vínculo `interface_modelo_id` para rastreabilidade. A decisão foi tomada porque a alternativa — ler sempre do modelo — tornaria impossível personalizar um equipamento e criaria efeitos colaterais em massa ao editar o catálogo.

Por que a região aponta para si mesma?

Porque a hierarquia é arbitrariamente profunda e não se sabe de antemão quantos níveis existirão. Uma tabela separada por nível (campus, prédio, sala) engessaria o modelo. O auto-relacionamento é a forma clássica de representar uma árvore em banco relacional — às vezes chamada de *adjacency list*. A contrapartida é que consultas de árvore inteira exigem recursão na aplicação, o que é aceitável para o volume atual.

Por que tipos são strings e não enums?

Os campos `tipo` (de equipamento, interface e porta) e `fluxo_ar` usam `String`. A vantagem é a flexibilidade: um novo tipo de equipamento ou de conector não exige alterar o schema nem rodar migração. A desvantagem é que o banco aceita qualquer texto; a padronização fica a cargo do formulário e da aplicação. Para o estágio atual, a flexibilidade pesou mais.

Por que guardar `dados_originais` em JSON?

O catálogo foi populado a partir de uma fonte externa, e cada modelo pode trazer dezenas de atributos que não têm coluna própria. Em vez de criar uma coluna para cada detalhe — e uma migração para cada novo detalhe —, o payload bruto é preservado em `dados_originais`. Isso permite auditoria e reprocessamento futuro sem alterar o schema. A desvantagem é que consultas dentro do JSON são menos legíveis e não têm a mesma tipagem de uma coluna dedicada.

Por que cascata automática no catálogo e manual no inventário?

No catálogo, a relação é de composição: uma interface de modelo não existe sem o modelo, então `delete-orphan` é adequado. No inventário, a exclusão de um equipamento tem efeitos externos — conexões que ligam a outras máquinas — e o projeto optou por tornar isso explícito no código: buscar conexões, apagar conexões, apagar portas, apagar equipamento, tudo em uma transação. O código fica mais longo, mas o comportamento é auditável e a mensagem de confirmação na interface avisa o usuário.

O que não existe no modelo

- **Timestamps** (`criado_em`, `atualizado_em`): não há histórico de quando cada registro foi criado ou alterado.
- **Usuário responsável**: como há um único login fixo, não faz sentido atribuir autoria.
- **Soft delete**: exclusões são físicas, não lógicas.
- **Constraints compostas** para nomes de interface e modelo por fabricante: a validação é feita na aplicação.
- **Índices explícitos** além dos criados pelas chaves primárias, estrangeiras e `unique`.

Nenhum desses itens impede o uso atual. Eles aparecem no capítulo 13 como melhorias priorizadas.

06 Rotas e blueprints

O sistema tem **28 rotas registradas**, distribuídas em cinco blueprints e no próprio `app.py`. Este capítulo apresenta o mapa completo e depois explica as funções de cada módulo, incluindo as validações que não aparecem no mapa.

6.1 O que é um blueprint, na prática

```
from flask import Blueprint

bp_fabricantes = Blueprint("fabricantes", __name__)


@bp_fabricantes.route("/fabricantes", methods=["GET", "POST"])
def fabricantes():
    ...
```

O blueprint é declarado com um nome lógico (`"fabricantes"`) e o módulo de importação (`__name__`). As rotas são decoradas com `@bp_fabricantes.route` em vez de `@app.route`. No `app.py`, `app.register_blueprint(bp_fabricantes)` conecta tudo.

A grande vantagem aparece nos templates: `url_for` usa o nome qualificado. Em vez de `url_for("fabricantes")`, escreve-se `url_for("fabricantes.fabricantes")`. Isso evita colisão entre funções com o mesmo nome em módulos diferentes e deixa claro de qual área a rota veio.

6.2 Mapa completo das rotas

MÉTODO	URL	FUNÇÃO	TEMPLATE / RESPOSTA
GET POST	`/login`	`login`	`login` ou redirect
GET	`/logout`	`logout`	redirect para `/login`
GET	`/`	`index`	`index`
GET	`/teste-banco`	`teste_banco`	texto simples
GET POST	`/regiao`	regioes `regiao`	`regiao`
GET POST	`/regiao/<int:id>/editar`	regioes `editar_regiao`	`editar_regiao`
POST	`/regiao/<int:id>/excluir`	regioes `excluir_regiao`	redirect
GET POST	`/fabricantes`	fabricantes `fabricantes`	`fabricantes`

MÉTODO	URL	FUNÇÃO	TEMPLATE / RESPOSTA
GET POST	`/fabricantes/<int:id>/editar`	fabricantes `editar_fabricante`	`editar_fabricante`
POST	`/fabricantes/<int:id>/excluir`	fabricantes `excluir_fabricante`	redirect
GET POST	`/modelos`	modelos `modelos`	`modelos`
GET POST	`/modelos/<int:id>/editar`	modelos `editar_modelo`	`editar_modelo`
POST	`/modelos/<int:id>/excluir`	modelos `excluir_modelo`	redirect
GET POST	`/modelos/<int:modelo_id>/interfaces`	modelos `interfaces_modelo`	`interfaces_modelo`
GET POST	`/modelos/<int:modelo_id>/interfaces/<int:interface_id>/editar`	modelos `editar_interface_modelo`	`editar_interface_modelo`
POST	`/modelos/<int:modelo_id>/interfaces/<int:interface_id>/excluir`	modelos `excluir_interface_modelo`	redirect
GET POST	`/modelos/<int:modelo_id>/portas-alimentacao`	modelos `portas_alimentacao_modelo`	`portas_alimentacao_modelo`
GET POST	`/modelos/<int:modelo_id>/portas-alimentacao/<int:porta_id>/editar`	modelos `editar_porta_alimentacao_modelo`	`editar_porta_alimentacao_modelo`
POST	`/modelos/<int:modelo_id>/portas-alimentacao/<int:porta_id>/excluir`	modelos `excluir_porta_alimentacao_modelo`	redirect
GET POST	`/equipamentos`	equipamentos `equipamentos`	`equipamentos`
GET	`/equipamentos/<int:id>`	equipamentos `detalhe_equipamento`	`detalhe_equipamento`
GET POST	`/equipamentos/<int:id>/editar`	equipamentos `editar_equipamento`	`editar_equipamento`
POST	`/equipamentos/<int:id>/excluir`	equipamentos `excluir_equipamento`	redirect
GET	`/equipamentos/<int:equipamento_id>/portas`	equipamentos `portas_equipamento`	`portas_equipamento`
GET	`/api/fabricantes/<int:fabricante_id>/modelos`	equipamentos `modelos_por_fabricante`	JSON

MÉTODO	URL	FUNÇÃO	TEMPLATE / RESPOSTA
GET POST	`/conexoes`	conexoes `conexoes`	`conexoes`
GET POST	`/nova-conexao/<int:interface_id>`	conexoes `nova_conexao`	`nova_conexao`
POST	`/desconectar/<int:conexao_id>`	conexoes `desconectar_cabo`	redirect

Na coluna "Template / resposta", a extensão `.html` dos templates foi omitida para manter a tabela legível.

> **POR QUE QUASE TUDO USA GET + POST NA MESMA URL?**
>
> Porque a URL de listagem também é a URL de criação. O GET mostra a lista e o formulário; o POST processa o formulário. É um padrão enxuto, comum em aplicações server-rendered, que evita uma rota extra só para receber o cadastro. A edição e a exclusão, por serem operações sobre um registro específico, ficam em URLs próprias.

6.3 Regiões: hierarquia, filtro e prevenção de ciclos

O módulo de regiões tem três funções e concentra a lógica mais interessante do ponto de vista algorítmico.

Listagem e montagem do seletor

A rota busca todas as regiões e as raízes (`regiao_pai_id = None`). Depois, uma função recursiva percorre a árvore e monta uma lista achatada com o nível de cada item:

```
def organizar_regioes(lista_regioes, nivel=0):
    for item in sorted(lista_regioes, key=lambda r: r.nome.lower()):
        regioes_para_selecao.append({
            "id": item.id,
            "nome": item.nome,
            "nivel": nivel
        })

        organizar_regioes(item.subregioes, nivel + 1)
```

O template usa `nivel` para adicionar recuo visual no `<option>`, de modo que o usuário enxergue a hierarquia mesmo em um elemento nativo de seleção. A ordenação é feita por `nome.lower()` para que maiúsculas e minúsculas não separem itens do mesmo bloco.

Filtro por subárvore

Quando o usuário escolhe uma região no filtro, a rota monta o conjunto de IDs visíveis começando pela região escolhida e incluindo recursivamente todas as suas sub-regiões. A árvore exibida passa a ter a região escolhida como raiz. Isso permite responder "mostre tudo o que está dentro do campus X", sem perder a estrutura.

Prevenção de ciclos na edição

Se a região A é pai de B, e B é pai de C, não é permitido definir C como pai de A: isso criaria um ciclo e tornaria a árvore infinita. A rota sobe pela cadeia de ancestrais da nova região pai e, se encontrar a própria região editada, recusa a operação:

```
verificar = regiao_pai

while verificar is not None:
    if verificar.id == regiao_atual.id:
        flash("Não é possível criar esse relacionamento, "
              "pois ele formaria um ciclo.", "erro")
        return redirect(url_for("regioes.editar_regiao", id=id))

    verificar = verificar.regiao_pai
```

O laço também cobre o caso mais simples, em que a região tenta ser pai dela mesma. Como o banco não tem uma constraint que impeça ciclos, essa validação é a única barreira — e é essencial.

Exclusão protegida

Uma região só pode ser excluída se não tiver sub-regiões e se não houver equipamentos vinculados. As duas verificações vêm antes do `delete`:

```
if regiao.subregioes:
    flash("Não é possível excluir: esta região possui sub-regiões.", "erro")
    return redirect(url_for("regioes.regiao"))

equipamentos_vinculados = Equipamento.query.filter_by(regiao_id=id).first()

if equipamentos_vinculados:
    flash("Não é possível excluir: existem equipamentos nesta região.", "erro")
    return redirect(url_for("regioes.regiao"))
```

Sem essas checagens, o PostgreSQL recusaria a exclusão por violação de chave estrangeira, mas o usuário veria um erro técnico. A validação na aplicação transforma o erro em uma explicação compreensível.

6.4 Fabricantes: o CRUD de referência

O CRUD de fabricantes é o mais simples e serve como modelo para os demais. A rota de listagem acumula as quatro operações de leitura e criação; a edição e a exclusão ficam separadas. As regras são:

- nome obrigatório, com `strip()` para remover espaços nas pontas;
- nome único, verificado antes do insert e reforçado pelo banco;
- exclusão bloqueada se existir qualquer modelo vinculado;
- `try/except` em toda escrita, com `rollback()` e registro no log via `current_app.logger.exception()`.

```
if fabricante.modelos:
    flash(
        "Não é possível excluir: existem modelos vinculados "
        "a este fabricante.",
        "erro"
    )
    return redirect(url_for("fabricantes.fabricantes"))
```

O padrão de erro é sempre o mesmo: mensagem amigável para o usuário (`flash`) e detalhe técnico para o desenvolvedor (`logger.exception`). Essa separação evita expor stack traces na interface.

6.5 Modelos, interfaces e portas de alimentação

O arquivo `rotas/modelos.py` é o maior do projeto e cobre três entidades: o modelo, suas interfaces e suas portas de alimentação. Ele é o melhor exemplo de validação centralizada e de paginação.

A validação compartilhada

Em vez de repetir as regras no cadastro e na edição, o módulo define `_validar_dados_modelo(modelo_id=None)`. Ela devolve uma tupla `(dados, erros)`: um dicionário pronto para virar objeto (ou para atualizar campos) e uma lista de mensagens. O parâmetro opcional `modelo_id` existe para a edição ignorar o próprio registro na checagem de unicidade.

```
def _validar_dados_modelo(modelo_id=None):
    """Valida o formulário e devolve (dados, erros)."""

    fabricante_id = request.form.get("fabricante_id", type=int)
    nome_modelo = request.form.get("modelo", "").strip()
    ...
    erros = []
    ...
    return dados, erros
```

As regras aplicadas são:

- **Altura em U:** converte texto em `float`, aceitando vírgula decimal (`"1,5"` vira `1.5`) e recusando valores negativos. Campo vazio vira `0.0`.
- **Peso:** mesma conversão, mas vazio vira `None`, porque peso desconhecido é diferente de peso zero.
- **Fabricante:** obrigatório e precisa existir no banco.
- **Nome do modelo:** obrigatório e único (ignorando o próprio registro na edição).
- **Profundidade completa:** é um checkbox; o valor é `"profundidade_completa" in request.form`, que resulta em booleano.

> **POR QUE ACEITAR VÍRGULA?**
>
> Porque o usuário brasileiro digita medidas como `1,5`. Se o sistema aceitasse apenas ponto, o formulário falharia silenciosamente para quem usa a convenção local. A conversão com `replace(",", ".")` resolve isso sem mudar o tipo do banco.

Paginação e filtro

A listagem de modelos usa `paginate(page=pagina, per_page=25, error_out=False)`. O `error_out=False` é importante: se o usuário pedir uma página inexistente, o Flask não devolve 404; ele retorna uma página vazia, que o template sabe exibir. O filtro por fabricante é aplicado antes da paginação, e a ordenação é por nome do fabricante e depois por nome do modelo.

```
modelos_paginados = (
    consulta
    .join(Fabricante)
    .order_by(Fabricante.nome, ModeloEquipamento.modelo)
    .paginate(page=pagina, per_page=25, error_out=False)
)
```

O `join` é necessário porque a ordenação usa uma coluna de outra tabela. Sem ele, o SQLAlchemy não saberia como relacionar as duas tabelas na consulta.

Interfaces do modelo

A rota de interfaces valida nome e tipo obrigatórios e impede nomes repetidos dentro do mesmo modelo. Na edição, a checagem exclui a própria interface da comparação. A listagem pagina de 50 em 50, coerente com o volume de 117 mil registros. A exclusão é bloqueada se a interface estiver sendo usada por algum equipamento:

```
if InterfaceEquipamento.query.filter_by(
    interface_modelo_id=interface.id
).first():
    flash(
        "Não é possível excluir: esta interface está sendo usada "
        "por equipamentos.",
        "erro"
    )
```

Portas de alimentação

O CRUD é semelhante ao de interfaces, mas sem paginação: o volume por modelo é pequeno (em geral uma ou duas fontes). A unicidade do nome é validada por modelo e a exclusão não tem dependências no inventário.

Exclusão de modelo

A exclusão de um modelo é a operação mais delicada do catálogo. Antes de apagar, a rota verifica dois níveis de dependência:

1. Existe algum `Equipamento` usando esse modelo? Se sim, bloqueia.
2. Alguma interface do modelo está sendo usada por `InterfaceEquipamento`? Se sim, bloqueia.

Só depois dessas verificações o `delete` é executado. Nesse momento, o `cascade="all, delete-orphan"` remove automaticamente as interfaces e portas de alimentação que pertencem ao modelo. É um exemplo de cascata *dentro* do catálogo, sem efeito no inventário.

6.6 Equipamentos: instanciação, cópia de portas e API

O módulo de equipamentos tem seis rotas e é o ponto em que o catálogo se transforma em inventário. A criação é o fluxo mais rico do sistema.

Validação e coerência fabricante/modelo

A rota exige nome, tipo e região. Fabricante e modelo são opcionais, mas, se ambos forem informados, precisam ser compatíveis. A regra é:

```
if modelo is None:
    fabricante = modelo.fabricante
elif modelo.fabricante_id ≠ fabricante.id:
    flash("Modelo incompatível com o fabricante.", "erro")
    return redirect(url_for("equipamentos.equipamentos"))
```

Se o usuário escolher apenas o modelo, o fabricante é derivado dele. Se escolher os dois e eles não combinarem, a operação é recusada. Isso impede um estado inconsistente que o banco, sozinho, não conseguiria detectar: as duas chaves estrangeiras existem, mas a combinação é semanticamente inválida.

O papel do flush()

```
db.session.add(novo_equipamento)
db.session.flush()

# CREATE - Cria as interfaces do modelo
if modelo is not None:
    for interface in modelo.interfaces:
        if not interface.habilitada:
            continue

        nova_interface = InterfaceEquipamento(
            equipamento_id=novo_equipamento.id,
            interface_modelo_id=interface.id,
            nome=interface.nome,
            tipo=interface.tipo,
            habilitada=True
        )

        db.session.add(nova_interface)

db.session.commit()
```

O `flush()` envia o INSERT do equipamento ao banco dentro da transação, mas **não confirma**. Isso faz o banco gerar o `id` do equipamento, que é necessário para criar as interfaces filhas. O `commit()` no final confirma tudo de uma vez: se algo falhar no meio, o `rollback()` desfaz o equipamento e as portas, evitando um equipamento sem portas.

A cópia ignora interfaces com `habilitada=False`. A lógica é: se o catálogo marca uma porta como desabilitada, ela não deve nascer no equipamento. As portas copiadas nascem sempre habilitadas, porque o estado físico é do equipamento, não do modelo.

Listagem com filtro por região

A listagem aceita `?filtro_regiao_id=` e aplica `filter_by(regiao_id= ... )` antes de ordenar por nome. O template mostra o formulário de filtro escondido atrás de um botão, controlado por JavaScript. É uma decisão de interface para não poluir a tela com um filtro que nem sempre é usado.

Detalhes, edição e exclusão

A rota de detalhes busca o equipamento por `db.session.get` e renderiza seus dados. A edição permite alterar nome, tipo e região — não permite trocar fabricante ou modelo, porque isso mudaria a planta de portas e exigiria decidir o que fazer com as conexões existentes. A exclusão é o fluxo em cascata manual descrito no capítulo 7.

A API de modelos por fabricante

Para que o formulário carregue os modelos sem recarregar a página, existe uma rota que devolve JSON:

```
@bp_equipamentos.route("/api/fabricantes/<int:fabricante_id>/modelos")
def modelos_por_fabricante(fabricante_id):

    fabricante = db.session.get(Fabricante, fabricante_id)

    if fabricante is None:
        return {"erro": "Fabricante não encontrado."}, 404

    modelos = ModeloEquipamento.query.filter_by(
        fabricante_id=fabricante_id
    ).order_by(ModeloEquipamento.modelo).all()

    return {
        "modelos": [
            {"id": modelo.id, "nome": modelo.modelo}
            for modelo in modelos
        ]
    }
```

O retorno é um dicionário Python; o Flask o serializa como JSON automaticamente. O status 404 é devolvido junto com a mensagem de erro quando o fabricante não existe. O JavaScript do template consome essa rota e preenche o `<select>` de modelos.

Portas do equipamento e o mapa de conexões

A rota de portas busca todas as interfaces do equipamento, ordena por nome e, para cada uma, consulta se existe conexão. O resultado é um dicionário `conexoes_portas` indexado pelo ID da porta, com o nome do outro equipamento, o nome da outra porta e o tipo de cabo. O template usa esse dicionário para decidir entre "Conectar cabo", "Desconectar" e "Indisponível".

> **CONSULTA POR PORTA (N+1)**
>
> A implementação atual faz uma consulta de conexão para cada porta. Com um equipamento de 48 portas, são 48 consultas pequenas. Funciona bem nesse volume, mas não escala para centenas de equipamentos listados ao mesmo tempo. A alternativa seria uma única consulta com `IN` ou um relacionamento configurado no model. É uma melhoria de desempenho, não um erro funcional.

6.7 Conexões: as regras que protegem o cabo

O módulo de conexões tem três rotas: a listagem/criação direta, o assistente em duas etapas (`/nova-conexao`) e a desconexão. A criação passa por uma sequência de validações antes de tocar no banco.

VALIDAÇÃO	MENSAGEM E MOTIVO
Origem e destino preenchidos	"Selecione as duas portas." — sem as duas pontas não há cabo.
Portas diferentes	"Escolha duas portas diferentes." — uma porta não conecta em si mesma.
Portas existentes	"Uma das portas selecionadas não existe." — protege contra IDs forjados.
Portas habilitadas	"As duas portas precisam estar habilitadas." — porta desabilitada não recebe cabo.
Equipamentos diferentes	"Selecione portas de equipamentos diferentes." — evita loop local sem sentido físico.
Tipo de cabo informado	"Informe o tipo de cabo." — o campo tem *default*, mas pode chegar vazio.
Portas ainda livres	"Uma das portas já está conectada." — a checagem busca qualquer conexão em que a porta apareça como origem ou destino.

A checagem de porta ocupada é feita com um único `or_` sobre as duas colunas:

```
conexao_existente = Conexao.query.filter(
    db.or_(
        Conexao.interface_origem_id.in_([origem_id, destino_id]),
        Conexao.interface_destino_id.in_([origem_id, destino_id])
    )
).first()
```

Isso cobre os quatro casos possíveis: a porta está como origem, como destino, ou ambas as portas já estão ocupadas em qualquer papel. Se qualquer combinação existir, a conexão é recusada.

O assistente em duas etapas

O fluxo `/nova-conexao/<interface_id>` é mais guiado. No primeiro GET, o usuário escolhe o equipamento de destino. O formulário envia `?equipamento_destino_id=...` por GET, e a rota carrega apenas as portas habilitadas e livres daquele equipamento. No POST, valida se a porta escolhida está de fato na lista de portas válidas — uma proteção contra manipulação do formulário — e só então grava.

```
if (porta_destino is None or porta_destino not in portas_destino):
    flash("Selecione uma porta de destino válida e disponível.", "erro")
```

Depois de conectar, o usuário volta para a página de portas do equipamento de origem. É um fluxo mais longo, porém mais seguro e compreensível do que escolher entre centenas de portas em um único `<select>`.

Desconexão

A rota de desconexão recebe o ID da conexão, guarda o equipamento de origem para saber para onde voltar, apaga a conexão e confirma. Como a conexão é o próprio cabo, apagá-la libera as duas portas automaticamente.

6.8 Login e logout

A autenticação foi implementada diretamente em `app.py`, sem blueprint e sem tabela de usuários. As credenciais são duas constantes: `USUARIO_PADRAO = "admin"` e `SENHA_PADRAO = "netlab"`. O login compara os valores, grava o usuário na sessão e redireciona.

O destino do redirecionamento vem de `next`, mas passa por uma checagem para evitar *open redirect*:

```
destino = (
    request.form.get("next")
    or request.args.get("next")
    or url_for("index")
)

# Evita redirecionar para fora do próprio site.
if not destino.startswith("/") or destino.startswith("//"):
    destino = url_for("index")

return redirect(destino)
```

Sem essa checagem, um link como `/login?next=https://site-malicioso` faria o usuário ser levado para fora do NetLab logo após entrar. A regra aceita apenas caminhos internos que começam com uma barra e não começam com duas.

O logout limpa toda a sessão com `session.clear()` e volta ao login. Como o `before_request` exige `session["usuario"]`, qualquer página interna deixa de responder após o logout.

07 Fluxos completos, passo a passo

Este capítulo mostra o sistema em movimento. Cada fluxo indica o que o usuário faz, qual rota é chamada, o que acontece no banco e o que o template exibe. É a melhor forma de entender como as peças dos capítulos anteriores se encaixam.

7.1 Montar o catálogo: fabricante → modelo → portas

O catálogo é construído de cima para baixo, porque cada nível depende do anterior. Não é possível cadastrar um modelo sem fabricante, nem uma interface sem modelo.

Figura 7.1 — Ordem de cadastro do catálogo

```
1. Fabricante
      POST /fabricantes  { nome: "Cisco" }
      └─ cria Fabricante(id=1, nome="Cisco")

2. Modelo
      POST /modelos  { fabricante_id: 1, modelo: "Catalyst 2960",
                       altura_u: "1", peso: "3,6", unidade_peso: "kg" }
      └─ valida fabricante, valida número, valida unicidade
      └─ cria ModeloEquipamento(id=10, fabricante_id=1, ...)

3. Interfaces do modelo
      POST /modelos/10/interfaces  { nome: "Gi0/1", tipo: "Ethernet",
                                     habilitada: on }
      └─ repete para Gi0/2 ... Gi0/24
      └─ cada uma vira InterfaceModelo(modelo_id=10, ...)

4. Portas de alimentação
      POST /modelos/10/portas-alimentacao  { nome: "PSU 1", tipo: "AC" }
      └─ cria PortaAlimentacaoModelo(modelo_id=10, ...)
```

No exemplo acima, o modelo tem 24 interfaces de rede e uma fonte. O catálogo está pronto para ser usado por qualquer equipamento. Nenhuma porta física existe ainda: elas só nascem quando um equipamento é instanciado.

7.2 Criar a hierarquia de regiões

A região raiz é criada sem pai. Depois, cada nível aponta para o anterior:

Figura 7.2 — Exemplo de árvore de regiões

```
POST /regiao  { nome: "Campus Centro", regiao_pai_id: "" }
   └─ Campus Centro (raiz)

POST /regiao  { nome: "Bloco A", regiao_pai_id: 1 }
   └─ Bloco A → Campus Centro

POST /regiao  { nome: "Sala 204", regiao_pai_id: 2 }
   └─ Sala 204 → Bloco A

Resultado exibido:

Campus Centro
  └─ Bloco A
       └─ Sala 204
```

O formulário mostra todas as regiões em um seletor com recuo por nível. O filtro da listagem permite escolher “Bloco A” e ver apenas ele e seus descendentes. Na edição, o sistema impede mover uma região para dentro da própria subárvore, como explicado no capítulo 6.

7.3 Cadastrar um equipamento e copiar as portas

Este é o fluxo que conecta catálogo e inventário. Considere que o modelo “Catalyst 2960” tem 24 interfaces habilitadas e uma desabilitada (a porta de console, por exemplo). O usuário preenche:

Figura 7.3 — Cadastro de equipamento

```
Tela: /equipamentos
  Nome:        SW-LAB-01
  Tipo:        Switch
  Região:      Sala 204
  Fabricante:  Cisco
  Modelo:      Catalyst 2960

Navegador → POST /equipamentos
  │
  ├─ valida nome, tipo e região
  ├─ valida fabricante e modelo (e a compatibilidade entre eles)
  │
  ├─ INSERT equipamentos → id = 1
  │     (flush para obter o id dentro da transação)
  │
  ├─ para cada InterfaceModelo habilitada do modelo:
  │     INSERT interfaces_equipamento
  │       equipamento_id = 1
  │       interface_modelo_id =
  │       nome, tipo copiados
  │       habilitada = true
  │
  └─ COMMIT
        │
        ▼
  flash("Equipamento cadastrado com sucesso!", "sucesso")
  redirect → GET /equipamentos
```

O resultado é um equipamento com 24 portas físicas, todas disponíveis. A porta desabilitada no modelo não foi copiada. Se o usuário abrir `/equipamentos/1/portas`, verá a lista com o estado "Disponível" em cada uma e o botão "Conectar cabo".

> **POR QUE A CÓPIA ACONTECE NO CADASTRO, E NÃO NA LEITURA?**
>
> Porque o equipamento precisa ter existência própria. Se as portas fossem sempre lidas do modelo, não seria possível desabilitar uma porta específica, registrar uma conexão sem alterar o catálogo, ou manter o histórico correto quando o modelo fosse editado. A cópia é o que transforma "planta" em "instalação".

7.4 Conectar um cabo

Há dois caminhos para conectar. O primeiro é a página de portas do equipamento, que já conhece a porta de origem. O segundo é a página geral de conexões, em que o usuário escolhe as duas pontas.

Caminho guiado

Figura 7.4 — Conectar a partir da porta de origem

```
Tela: /equipamentos/1/portas
  Porta Gi0/1 → botão "Conectar cabo"
  │
  ▼
GET /nova-conexao/
  │  verifica se a porta existe, está habilitada e está livre
  │  lista os outros equipamentos
  │
  ▼
Usuário escolhe o equipamento de destino
  │
  ▼
GET /nova-conexao/?equipamento_destino_id=2
  │  lista apenas portas habilitadas e livres do destino
  │
  ▼
Usuário escolhe a porta, o tipo de cabo e envia o POST
  │
  ├─ valida se a porta de destino está na lista permitida
  ├─ cria Conexao(origem, destino, tipo_cabo, descricao)
  ├─ COMMIT
  │
  ▼
flash("Cabo conectado com sucesso!", "sucesso")
redirect → /equipamentos/1/portas
```

Depois da conexão, a porta de origem passa a exibir “Conectada” e o nome do equipamento e da porta do outro lado. A porta de destino, se consultada, mostra a mesma conexão vista do outro extremo — porque a consulta considera a porta tanto como origem quanto como destino.

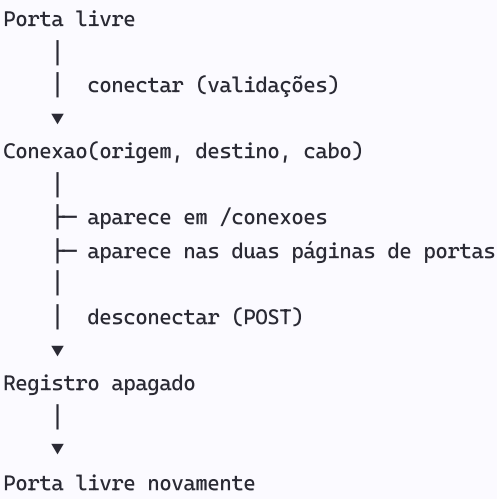
Caminho direto

Em `/conexoes`, o usuário vê dois seletores com todas as portas habilitadas e escolhe origem e destino. O POST passa pelas sete validações listadas no capítulo 6 antes de gravar. Esse caminho é mais rápido para quem já sabe exatamente quais portas usar, mas é mais sujeito a erro humano — por isso as validações são ainda mais importantes.

7.5 Desconectar um cabo

Na página de portas, uma porta conectada mostra o botão “Desconectar”. O formulário envia um POST para `/desconectar/<id_da_conexao>` com confirmação do navegador. A rota busca a conexão, guarda o equipamento de origem, apaga o registro e redireciona de volta. As duas portas voltam ao estado “Disponível” imediatamente, porque a conexão deixou de existir.

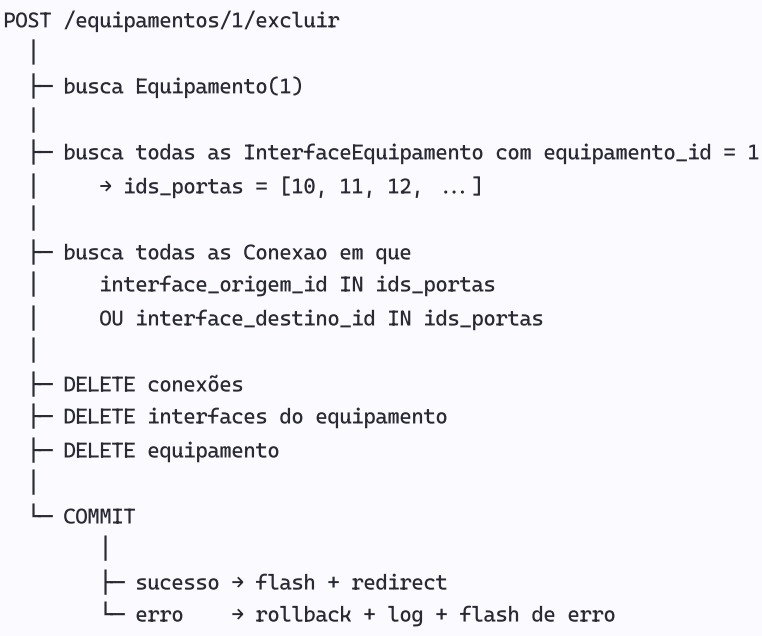
Figura 7.5 — Ciclo de vida de uma conexão



7.6 Excluir um equipamento em cascata

A exclusão de equipamento é a operação com mais efeitos colaterais. O código não confia em cascatas automáticas do banco: ele busca e apaga em ordem, dentro de uma única transação.

Figura 7.6 — Ordem da exclusão



A ordem importa. Se o equipamento fosse apagado primeiro, as interfaces ficariam com uma chave estrangeira inválida e o banco recusaria a operação. Se as interfaces fossem apagadas antes das conexões, as conexões apontariam para portas inexistentes. O código resolve as duas dependências na ordem correta.

O template pede confirmação com uma mensagem explícita: “Deseja realmente excluir este equipamento e suas conexões?”. A confirmação é do navegador, não do servidor; ela melhora a experiência, mas não substitui a validação no backend.

7.7 Regras de integridade consolidadas

A tabela abaixo reúne todas as regras que o sistema aplica, onde cada uma é verificada e o que acontece se for violada. Ela serve como referência rápida para manutenção.

REGRA	ONDE É APLICADA	CONSEQUÊNCIA
Nome do fabricante único	Rota + `unique` no banco	Mensagem amigável ou erro de constraint em corrida.
Nome do modelo único	Rota + `unique` no banco	Cadastro recusado com mensagem.
Modelo pertence ao fabricante	`rotas/equipamentos.py`	Cadastro de equipamento recusado.
Região pai existe	`rotas/regioes.py`	Cadastro/edição recusados.
Sem ciclo na hierarquia	`rotas/regioes.py` (edição)	Edição recusada com explicação.
Sem sub-regiões ao excluir	`rotas/regioes.py`	Exclusão bloqueada.
Sem equipamentos na região ao excluir	`rotas/regioes.py`	Exclusão bloqueada.
Sem modelos ao excluir fabricante	`rotas/fabricantes.py`	Exclusão bloqueada.
Sem equipamentos ao excluir modelo	`rotas/modelos.py`	Exclusão bloqueada.
Interface de modelo não usada ao excluir	`rotas/modelos.py`	Exclusão bloqueada.
Nome de interface único por modelo	`rotas/modelos.py`	Cadastro/edição recusados.
Nome de porta de alimentação único por modelo	`rotas/modelos.py`	Cadastro/edição recusados.
Duas portas diferentes e existentes	`rotas/conexoes.py`	Conexão recusada.
Portas habilitadas	`rotas/conexoes.py`	Conexão recusada.
Equipamentos diferentes	`rotas/conexoes.py`	Conexão recusada.
Porta participa de no máximo uma conexão	Rota + `unique` nas duas FKs	Conexão recusada.

REGRA DE OURO DO PROJETO

Toda operação de escrita segue o mesmo esqueleto: validar, tentar, confirmar, informar e redirecionar. Toda operação de escrita também tem um caminho de erro com `rollback()` e log. Se você memorizar esse esqueleto, consegue ler qualquer rota do NetLab.

08 Templates e interface

O projeto tem **18 templates**, um para cada tela ou variação de formulário. Eles são HTML comum com sintaxe Jinja, recebem os dados via `render_template` e são servidos prontos pelo Flask. Não há componentização no navegador nem etapa de build.

8.1 Organização dos templates

TEMPLATE	PAPEL
`index.html`	Página inicial com quatro cartões de acesso rápido.
`login.html`	Tela de autenticação, fora do layout interno.
`regiao.html`	Cadastro, filtro e árvore de regiões.
`editar_regiao.html`	Edição de nome, descrição e região pai.
`fabricantes.html`	Cadastro e listagem de fabricantes.
`editar_fabricante.html`	Edição do nome do fabricante.
`modelos.html`	Cadastro, filtro e listagem paginada de modelos.
`editar_modelo.html`	Edição dos dados técnicos do modelo.
`interfaces_modelo.html`	Cadastro e listagem paginada de interfaces do modelo.
`editar_interface_modelo.html`	Edição de nome, tipo e flags da interface.
`portas_alimentacao_modelo.html`	Cadastro e listagem de fontes do modelo.
`editar_porta_alimentacao_modelo.html`	Edição da porta de alimentação.
`equipamentos.html`	Cadastro, filtro e listagem de equipamentos.
`detalhe_equipamento.html`	Ficha do equipamento com dados e atalhos.
`editar_equipamento.html`	Edição de nome, tipo e região.
`portas_equipamento.html`	Tabela de portas com ações de conexão.
`conexoes.html`	Listagem geral de cabos e criação direta.
`nova_conexao.html`	Assistente de conexão em duas etapas.

8.2 O padrão de página (e a ausência de um base.html)

Cada template repete a estrutura completa: `DOCTYPE`, `head`, link para o CSS, `header` com marca e navegação, `main`, mensagens e `footer`. Não existe `base.html` nem `{% extends %}`.

A repetição tem um custo: qualquer mudança no menu precisa ser aplicada em 15 arquivos (todos menos o login e os dois templates legados). Foi exatamente isso que aconteceu ao adicionar o link "Sair". A vantagem é que cada template é autossuficiente e fácil de ler isoladamente, o que ajuda quem está aprendendo.

> **REFATORAÇÃO RECOMENDADA**
>
> Criar `templates/base.html` com os blocos `{% block titulo %}`, `{% block conteudo %}` e um `{% block scripts %}`, e fazer os demais templates estenderem essa base. O ganho é grande e o risco é baixo, mas a mudança deve ser feita com calma, um template por vez, testando cada rota.

8.3 Jinja na prática

Variáveis e url_for

```
<h1>{{ equipamento.nome }}</h1>

<a href="{{ url_for('equipamentos.detalhe_equipamento', id=equipamento.id) }}">
    Detalhes
</a>
```

`url_for` gera a URL a partir do nome da função e dos parâmetros. Isso evita concatenar strings e mantém os links corretos mesmo se a rota mudar. Em blueprints, o nome é qualificado: `equipamentos.detalhe_equipamento`.

Laços e condicionais

```
{% for equipamento in equipamentos %}
    <tr>
        <td>{{ equipamento.nome }}</td>
        <td>
            {% if equipamento.fabricante %}
                {{ equipamento.fabricante.nome }}
            {% else %}
                —
            {% endif %}
        </td>
    </tr>
{% else %}
    <tr><td colspan="2">Nenhum equipamento cadastrado.</td></tr>
{% endfor %}
```

O `{% else %}` do laço Jinja é executado quando a lista está vazia. Ele evita a duplicação de um bloco `if not equipamentos`.

Mensagens flash

Todas as páginas internas usam o mesmo bloco para exibir mensagens. As categorias `sucesso` e `erro` viram classes CSS:

```
{% with mensagens = get_flashed_messages(with_categories=true) %}
    {% if mensagens %}
        {% for categoria, mensagem in mensagens %}
            <div class="mensagem {{ categoria }}">{{ mensagem }}</div>
        {% endfor %}
    {% endif %}
{% endwith %}
```

O `with` limita o escopo da variável e consome as mensagens da sessão. Como o padrão é POST → redirect → GET, a mensagem aparece uma única vez, na página seguinte, e desaparece no próximo carregamento.

Macro recursiva na árvore de regiões

O template `regiao.html` define uma macro que se chama para cada sub-região. É a contraparte visual da recursão feita na rota:

```
{% macro exibir_regiao(item, nivel=0) %}
    ...
    {% for filha in item.subregioes|sort(attribute='nome') %}
        {{ exibir_regiao(filha, nivel + 1) }}
    {% endfor %}
{% endmacro %}
```

A macro usa `nivel` para recuo e verifica `item.id in ids_regioes_visiveis` para respeitar o filtro. O `sort(attribute='nome')` mantém a ordem alfabética em cada nível. É um bom exemplo de como Jinja consegue expressar lógica de árvore sem JavaScript.

8.4 JavaScript: o mínimo necessário

Há dois comportamentos dinâmicos no projeto, ambos em `equipamentos.html`.

Carregar modelos conforme o fabricante

O select de modelo começa desabilitado, com o texto "Selecione primeiro um fabricante". Quando o fabricante muda, o script faz um `fetch` para a API, limpa o select, adiciona a opção "Selecione um modelo" e insere um `Option` para cada item retornado.

```
const resposta = await fetch(
    `/api/fabricantes/${fabricanteId}/modelos`
);

if (!resposta.ok) {
    throw new Error("Falha ao consultar modelos");
}

const dados = await resposta.json();

seletorModelo.innerHTML = "";
seletorModelo.add(new Option("Selecione um modelo", ""));

for (const modelo of dados.modelos) {
    seletorModelo.add(new Option(modelo.nome, modelo.id));
}
```

O bloco `try/catch/finally` garante que o select seja reabilitado mesmo em caso de erro, e o `finally` é o lugar certo para isso. Se o fabricante não tiver modelos, o select mostra "Nenhum modelo encontrado" em vez de ficar vazio sem explicação.

Mostrar e ocultar o filtro de região

O botão "Filtrar por região" alterna o atributo `hidden` do formulário e atualiza `aria-expanded`. O uso de `aria-expanded` é um detalhe de acessibilidade: leitores de tela anunciam se o conteúdo está aberto ou fechado. O formulário continua funcionando sem JavaScript? Não — ele começa oculto. Se o JavaScript falhar, o filtro fica inacessível. Uma melhoria seria deixá-lo visível por padrão e usar o script apenas para recolhê-lo.

8.5 O design system Termix

O visual atual foi inspirado no tema Termix: uma interface escura, monoespaçada e precisa, com acento roxo. O objetivo é afastar-se do "painel administrativo genérico" e dar ao NetLab a aparência de uma ferramenta de infraestrutura.

Tokens

TOKEN	VALOR	USO
`--fundo`	`#09090a`	Fundo geral, quase preto.
`--painel`	`#0d0d0f`	Cartões e superfícies elevadas.
`--borda`	`#242427`	Bordas finas de 1px.
`--texto`	`#e8e8ea`	Texto principal.
`--texto-suave`	`#7c7c85`	Rótulos e texto secundário.
`--roxo`	`#a855f7`	Acento, foco e estado ativo.

TOKEN	VALOR	USO
`--roxo-fundo`	`rgba(168,85,247,.10)`	Fundo de botões e etiquetas de destaque.
`--verde`	`#6ee7a0`	Sucesso e disponibilidade.
`--vermelho`	`#fb7185`	Erro, exclusão e desconexão.

Tipografia e ritmo

A fonte é `JetBrains Mono`, com fallback para Cascadia Mono, Consolas e monoespaçadas do sistema. O corpo tem 13px e entrelinha 1.65. Rótulos de formulário e *badges* usam 9 a 10px, com `letter-spacing` de 0.12 a 0.16em e `text-transform: uppercase`. É esse contraste entre texto pequeno em caixa alta e conteúdo legível que dá o tom de terminal.

Componentes

- **Botões:** três variantes — padrão, principal (roxo) e perigo (vermelho). Todos usam borda de 1px e fundo translúcido; o hover intensifica a borda e adiciona um brilho discreto.
- **Cartões:** fundo `--painel`, borda de 1px, raio de 4px e sombra suave. O cabeçalho do cartão tem uma linha inferior e título em caixa alta.
- **Tabelas:** cabeçalho escuro, células com borda inferior sutil, hover roxo quase imperceptível e primeira coluna em destaque.
- **Etiquetas:** `conectado` em roxo, `disponivel` em verde e o estado desabilitado em cinza.
- **Mensagens:** barra lateral colorida (roxo para informação, verde para sucesso, vermelho para erro) sobre fundo translúcido.
- **Login:** painel centralizado, com uma linha roxa no topo, dica de acesso e rodapé.

Layout responsivo

As páginas usam `width: min(1240px, 94%)` para manter margens em telas grandes e respiro em telas pequenas. A grade de duas colunas vira uma coluna em `760px`. Nas páginas de CRUD, a regra `.grade:has(table)` cria uma coluna estreita para o formulário e uma larga para a tabela; abaixo de `1100px`, volta a empilhar.

O uso de `:has()` é elegante porque dispensa uma classe extra no HTML: o CSS detecta a presença da tabela e ajusta a grade. O custo é que navegadores muito antigos não suportam a seletora; nos navegadores atuais (Chrome, Edge, Firefox e Safari) funciona bem.

8.6 Inconsistências visuais conhecidas

Dois templates ainda usam a estrutura antiga: `conexoes.html` e `nova_conexao.html`. Eles têm `<header>` simples, sem a classe `cabecalho`, sem o menu de navegação e sem o link “Sair”. Funcionam, mas quebram a consistência do tema.

A correção é direta: aplicar o mesmo cabeçalho e os mesmos cartões usados nas demais páginas, além de envolver o conteúdo em `class="conteudo"`. Como esses dois templates usam `container` e `card`, o CSS atual já os estiliza, mas o resultado não é idêntico ao restante do sistema.

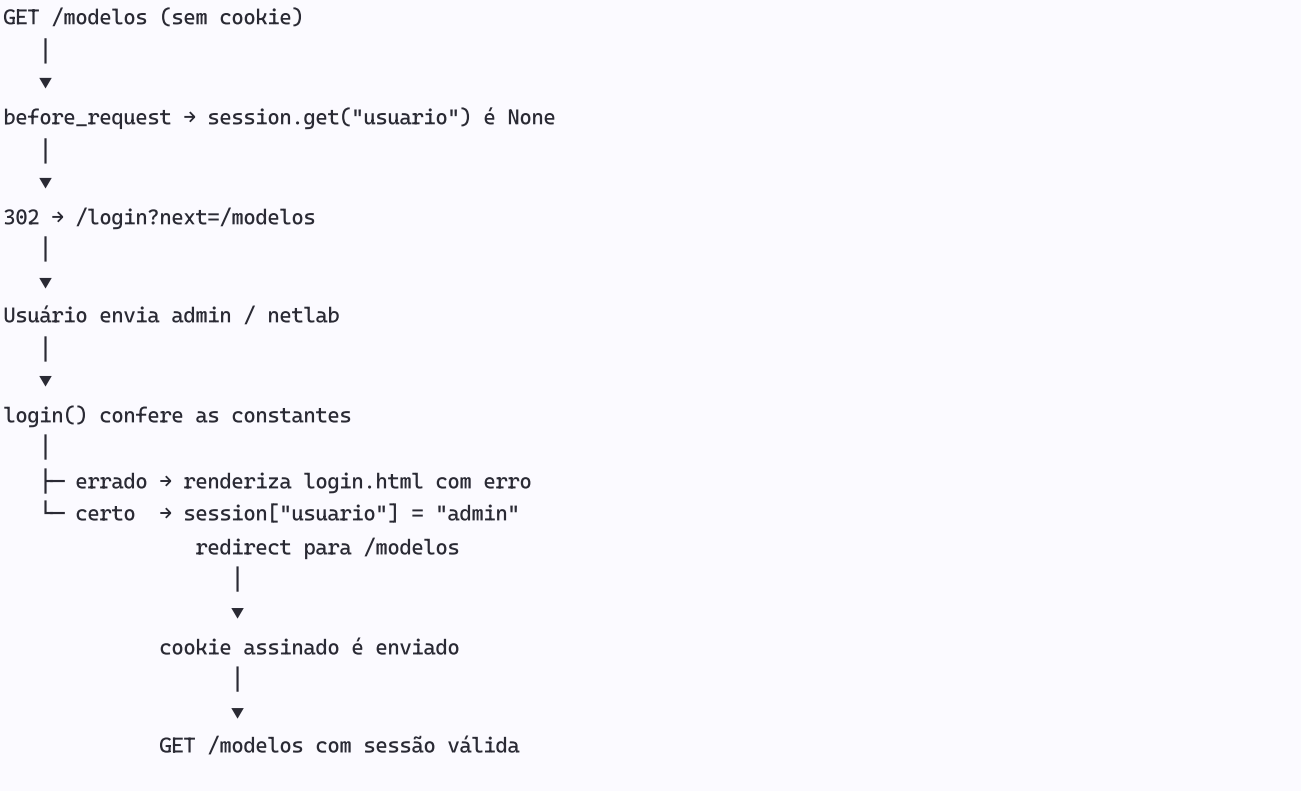
09 Login, sessão e segurança

O login foi pedido como "o mais simples possível, sem validações seguras", e foi implementado exatamente assim: duas credenciais fixas e uma sessão de Flask. Este capítulo explica como isso funciona e, principalmente, o que precisa mudar se o sistema deixar de ser local.

9.1 Como a sessão funciona

O Flask usa um cookie de sessão assinado com `SECRET_KEY`. Quando o usuário acerta usuário e senha, a rota grava `session["usuario"] = usuario`. O Flask serializa esse dicionário, assina com a chave e envia no cookie. Nas requisições seguintes, o navegador devolve o cookie e o Flask valida a assinatura antes de disponibilizar `session`.

Figura 9.1 — Fluxo de autenticação



O cookie é **assinado, não criptografado**. Isso significa que o usuário não consegue alterá-lo sem invalidar a assinatura, mas o conteúdo não é secreto. Por isso nunca se deve guardar senha ou dado sensível dentro da sessão. Aqui só há o nome do usuário.

9.2 O before_request como portaria

A proteção é global, não rota a rota. Qualquer nova rota criada no futuro já nasce protegida, a menos que seu `endpoint` seja explicitamente liberado. A lista de exceções é curta e intencional:

```
if request.endpoint in ("login", "static"):
    return None
```

`static` é necessário para que o CSS da própria tela de login seja carregado. `login` é necessário para que o usuário consiga se autenticar. Tudo o mais — inclusive `/teste-banco` — exige sessão.

9.3 Limitações conscientes

LIMITAÇÃO	RISCO E O QUE FAZER
Credenciais em texto puro no código	Quem tiver acesso ao repositório vê a senha. Mover para variáveis de ambiente e usar hash (por exemplo, `generate_password_hash` do Werkzeug).
Um único usuário fixo	Não há cadastro, troca de senha, perfis ou auditoria. Para múltiplos usuários, criar um model `Usuario` e uma tabela de sessões ou usar Flask-Login.
`SECRET_KEY` provisória	Se a chave vazar, alguém pode forjar cookies. Gerar uma chave forte e mantê-la fora do código.
Sem proteção CSRF	Um site externo poderia induzir o navegador a enviar um POST para exclusão. Adicionar Flask-WTF e um token nos formulários.
`debug=True`	O debugger interativo permite execução de código se exposto. Em produção, rodar com `debug=False` e um servidor WSGI.
Sem HTTPS	Em rede local, o cookie trafega em claro. Publicar apenas atrás de HTTPS e marcar o cookie como `Secure` e `HttpOnly`.
Sem limitação de tentativas	Não há proteção contra força bruta. Para uso local, é aceitável; para exposição, adicionar rate limiting.

9.4 Como endurecer sem reescrever tudo

Mover configuração para o ambiente

```
import os

app.config['SECRET_KEY'] = os.environ.get(
    'NETLAB_SECRET_KEY', 'chave-apenas-para-desenvolvimento'
)
app.config['SQLALCHEMY_DATABASE_URI'] = os.environ.get(
    'NETLAB_DATABASE_URL',
    'postgresql://postgres:4548@localhost:5432/netlab'
)
```

O sistema continua funcionando sem variáveis definidas, mas passa a aceitar configuração externa. Em produção, basta definir as variáveis e nunca commitar os valores reais.

Guardar hash em vez da senha

```
from werkzeug.security import generate_password_hash, check_password_hash

SENHA_HASH = generate_password_hash("netlab")

if usuario == USUARIO_PADRAO and check_password_hash(SENHA_HASH, senha):
    session["usuario"] = usuario
```

O hash não impede um ataque de força bruta, mas impede que a senha seja lida diretamente caso o código ou o banco vaze. O próximo passo natural é mover o hash para uma tabela de usuários.

CSRF nos formulários

Com Flask-WTF, basta inicializar `CSRFProtect(app)` e incluir `{{ csrf_token() }}` em cada formulário. Como todas as operações destrutivas já usam POST, a mudança é localizada e não altera a lógica de negócio.

> **PRIORIDADE REALISTA**
>
> Para um projeto acadêmico local, o login atual cumpre o objetivo. Se o NetLab for publicado, a ordem de prioridade é: HTTPS + chave secreta forte, hash de senha, CSRF e, por fim, usuários e perfis. Não é preciso fazer tudo de uma vez.

10 Como executar o projeto

Esta seção descreve o caminho do zero até a tela de login, considerando o ambiente Windows usado no desenvolvimento. Os comandos são de PowerShell.

10.1 Pré-requisitos

- **Python 3.14** (o ambiente de referência usa 3.14.6).
- **PostgreSQL** instalado e em execução na porta 5432.
- **Banco** `netlab` criado e acessível.
- Opcional: **pgAdmin** ou `psql` para inspecionar dados.

10.2 Criar o banco

No `psql`, conectado como o usuário configurado:

```
CREATE DATABASE netlab;
```

As tabelas não precisam ser criadas manualmente: o `db.create_all()` do `app.py` faz isso na primeira execução. Se o banco já existir com dados, ele apenas verifica o que falta.

10.3 Ambiente virtual e dependências

```
cd "C:\Users\Viktor\Documents\Faculdade\4° Período\Física\Projeto NetLab"

python -m venv .venv
.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
pip install -r requerimentos.txt
```

> **O ARQUIVO ESTÁ EM UTF-16**
>
> O arquivo de dependências se chama `requerimentos.txt` e está gravado em UTF-16. O `pip` espera UTF-8 e pode falhar ao lê-lo. Se isso acontecer, converta antes:
>
> ```
> Get-Content requerimentos.txt -Encoding Unicode |
>     Set-Content requirements.txt -Encoding UTF8
>
> pip install -r requirements.txt
> ```

As dependências declaradas são: Flask 3.1.3, Flask-SQLAlchemy 3.1.1, SQLAlchemy 2.1.4, psycopg 3.3.6, Jinja2 3.1.6, Werkzeug 3.1.9, entre outras de suporte (blinker, click, itsdangerous, MarkupSafe, typing_extensions, tzdata).

10.4 Conferir a conexão

A URL do banco fica em `app.py`. Se o seu PostgreSQL usar outra senha, outro usuário ou outra porta, ajuste esta linha:

```
app.config['SQLALCHEMY_DATABASE_URI'] = (
    'postgresql://postgres:4548@localhost:5432/netlab'
)
```

O formato é `postgresql://usuario:senha@host:porta/banco`. Se a senha tiver caracteres especiais, eles precisam ser codificados na URL (por exemplo, `@` vira `%40`).

10.5 Executar

```
python app.py
```

O Flask sobe em `http://127.0.0.1:5000`. A saída esperada é semelhante a:

```
 * Serving Flask app 'app'
 * Debug mode: on
 * Running on http://127.0.0.1:5000
 * Debugger is active!
```

Abra o endereço no navegador. A primeira tela será o login. Use:

```
Usuário: admin
Senha:   netlab
```

Depois de entrar, a página inicial mostra os quatro módulos. O link “Sair” fica no canto direito do menu.

10.6 Testar a conexão com o banco

Com a sessão ativa, acesse `/teste-banco`. A rota executa `SELECT 1` e responde com texto simples:

```
Conexao com PostgreSQL realizada com sucesso!
```

Se houver erro, a resposta traz a mensagem e o log registra o detalhe. As causas mais comuns são PostgreSQL parado, senha incorreta, banco inexistente ou driver não instalado.

10.7 Problemas comuns na execução

SINTOMA	CAUSA PROVÁVEL E SOLUÇÃO
`ModuleNotFoundError: No module named 'flask'`	Ambiente virtual não ativado ou dependências não instaladas. Ative o `.venv` e rode o `pip install`.

SINTOMA	CAUSA PROVÁVEL E SOLUÇÃO
`connection refused` na porta 5432	PostgreSQL parado ou em outra porta. Inicie o serviço e confira a URL.
`password authentication failed`	Usuário/senha diferentes do banco. Ajuste a URI.
`database "netlab" does not exist`	O banco não foi criado. Rode `CREATE DATABASE netlab;`.
`Address already in use`	Já existe um processo na porta 5000. Encerre-o ou rode com outra porta: `flask run --port 5001`.
Página em branco ou sem estilo	O CSS não carregou. Confira `/static/style.css` e o console do navegador.
Redirecionamento infinito para o login	Cookie de sessão bloqueado ou `SECRET_KEY` mudando a cada requisição. Verifique a configuração e o relógio do sistema.

10.8 Sobre execução em produção

O servidor embutido do Flask não é adequado para produção: ele é single-thread por padrão, não tem balanceamento e o modo debug expõe o depurador. No Windows, uma opção simples é o **Waitress**:

```
pip install waitress
waitress-serve --listen=0.0.0.0:8000 app:app
```

Antes disso, porém, aplique as melhorias de segurança do capítulo 9 e desligue o `debug`. Publicar o sistema como está serviria apenas para demonstração controlada.

11 Guia rápido de uso

Receitas curtas para as tarefas do dia a dia. Todas assumem que você já entrou no sistema.

11.1 Criar uma região com sub-regiões

1. Abra **Regiões** no menu.
2. Preencha o nome (obrigatório) e, se quiser, a descrição.
3. Deixe "Região pai" como "Nenhuma" para criar uma raiz.
4. Clique em **Cadastrar região**.
5. Para criar uma filha, selecione a região pai no mesmo formulário.

Para ver apenas uma parte da árvore, use o filtro no topo da listagem. Para mover uma região, clique em **Editar** e escolha outro pai — o sistema recusa movimentos que criariam ciclo.

11.2 Cadastrar um fabricante

1. Abra **Fabricantes**.
2. Digite o nome e clique em **Cadastrar**.
3. Nomes repetidos são recusados, inclusive com diferença apenas de espaços nas pontas.

11.3 Cadastrar um modelo completo

1. Abra **Modelos** e preencha fabricante, nome, altura em U, peso e unidade.
2. Marque "Profundidade completa" se o equipamento ocupar o rack inteiro.
3. Informe o fluxo de ar, se souber.
4. Clique em **Cadastrar modelo**.
5. Na listagem, clique em **Interfaces** para adicionar as portas de rede.
6. Clique em **Portas de alimentação** para registrar as fontes.

> **ALTURA EM U**
>
> Use ponto ou vírgula: `1`, `1.5` e `1,5` são aceitos. O valor representa quantas unidades de rack o equipamento ocupa. Deixe vazio se não souber; o sistema assume `0.0`.

11.4 Adicionar interfaces a um modelo

1. Na lista de modelos, clique em **Interfaces**.
2. Informe o nome (ex.: `Gi0/1`) e o tipo (ex.: `Ethernet`).
3. Marque **Habilitada** para que a porta seja copiada em novos equipamentos.
4. Marque **Gerenciamento** se for uma porta de administração.

5. Repita para cada porta. A lista pagina de 50 em 50.

11.5 Cadastrar um equipamento

1. Abra **Equipamentos**.
2. Informe o nome, escolha o tipo e a região.
3. Escolha o fabricante; o campo de modelo é habilitado e carregado via API.
4. Escolha o modelo e clique em **Cadastrar equipamento**.
5. Abra os detalhes e depois **Portas** para ver as interfaces criadas.

Se você escolher um modelo sem escolher fabricante, o sistema assume o fabricante do modelo. Se escolher os dois e eles não combinarem, o cadastro é recusado.

11.6 Conectar e desconectar cabos

1. Abra as portas do equipamento de origem.
2. Na porta desejada, clique em **Conectar cabo**.
3. Escolha o equipamento de destino e clique em **Mostrar portas disponíveis**.
4. Escolha a porta, o tipo de cabo, escreva uma descrição se quiser e confirme.
5. Para desconectar, volte às portas e clique em **Desconectar**.

A página **Conexões** oferece o caminho alternativo: escolher as duas portas de uma vez. Ela é útil para lançar vários cabos em sequência, mas exige mais atenção.

11.7 Excluir com segurança

A ordem natural é inversa à da criação: primeiro cabos, depois equipamentos, depois interfaces e portas do modelo, depois o modelo, o fabricante e, por fim, as regiões. O sistema bloqueia exclusões que deixariam dependências órfãs e explica o motivo na mensagem.

PARA EXCLUIR	ANTES É PRECISO
Região	Remover ou mover sub-regiões e equipamentos.
Fabricante	Excluir todos os modelos vinculados.
Modelo	Excluir equipamentos que o usam e liberar interfaces em uso.
Interface de modelo	Nenhum equipamento pode estar usando aquela interface.
Equipamento	Nada: a exclusão remove conexões e portas junto.

12 Manutenção e diagnóstico

Este capítulo é para quem vai mexer no código depois. Ele descreve o passo a passo para adicionar um CRUD, como alterar o schema com segurança e como interpretar os erros mais comuns.

12.1 Checklist para adicionar um novo CRUD

1. **Criar o model** em `models/`, com `__tablename__`, colunas e relacionamentos. Usar `nullable=False` no que é obrigatório e `unique=True` no que não pode repetir.
2. **Garantir que o model seja importado** por alguma rota ou pelo `app.py`. Sem isso, `create_all()` não cria a tabela.
3. **Criar o blueprint** em `rotas/`, com as rotas de listagem/criação, edição e exclusão.
4. **Registrar o blueprint** em `app.py` com `app.register_blueprint( ... )`.
5. **Criar os templates** de listagem e edição, seguindo o padrão de flash messages e `url_for` qualificado.
6. **Adicionar o link no menu** de todas as páginas (ou, melhor, criar o `base.html` e centralizar).
7. **Testar** criação, edição, exclusão, validações e o caminho de erro. Rodar um smoke test nas rotas existentes para garantir que nada quebrou.

> **TESTE RÁPIDO SEM NAVEGADOR**
>
> O Flask tem um cliente de testes que dispensa subir o servidor. O trecho abaixo autentica e verifica o status de várias rotas:

```
import app

app.app.config.update(TESTING=True)
cliente = app.app.test_client()
cliente.post("/login", data={"usuario": "admin", "senha": "netlab"})

for rota in ["/", "/modelos", "/equipamentos", "/fabricantes"]:
    resposta = cliente.get(rota)
    print(resposta.status_code, rota)
```

12.2 Alterações de schema: o que fazer e o que não fazer

Como o projeto não usa migrações, qualquer mudança estrutural exige atenção. O fluxo seguro é:

1. Fazer backup dos dados afetados (exportar CSV ou usar `pg_dump`).
2. Alterar o model.
3. Escrever e executar o `ALTER TABLE` correspondente no PostgreSQL.
4. Reiniciar a aplicação e testar as rotas que usam a tabela.

Exemplos de comandos:

```
-- adicionar uma coluna opcional
ALTER TABLE equipamentos ADD COLUMN patrimonio VARCHAR(80);

-- remover uma coluna
ALTER TABLE modelos_equipamento DROP COLUMN slug;

-- criar uma constraint de unicidade composta
ALTER TABLE modelos_equipamento
    ADD CONSTRAINT uq_modelo_por_fabricante
    UNIQUE (fabricante_id, modelo);
```

> **NUNCA CONFIE NO CREATE_ALL PARA ALTERAR**
>
> `db.create_all()` só cria o que não existe. Ele não compara o model com a tabela, não adiciona colunas, não muda tipos e não remove constraints. Acreditar que ele "sincroniza" o banco é a origem de muitos erros silenciosos.

12.3 O caso do campo slug

O projeto teve um campo `slug` no model de modelos de equipamento, usado para gerar identificadores amigáveis. Com o tempo, ele se mostrou desnecessário: o `id` já identifica o registro e o nome do modelo é único. O campo foi removido do model, das rotas, dos templates e do formulário.

A remoção do código não removeu a coluna do banco. Como o `create_all()` não altera tabelas existentes, a coluna `slug` continuou no PostgreSQL, agora sem uso. Foi preciso executar um `ALTER TABLE ... DROP COLUMN` e guardar um backup dos valores antes, em `backup_slug_modelos.csv`.

A lição é dupla: (1) remover um campo do código é só metade da remoção; (2) um resíduo no template — um `<script>` que referenciava o campo — pode quebrar o JavaScript da página mesmo que o backend esteja correto. Por isso, ao remover um campo, procure o nome dele em **models, rotas, templates, JavaScript e banco**.

12.4 Erros comuns e como diagnosticar

ERRO	SIGNIFICADO E SOLUÇÃO
`relation "x" does not exist`	A tabela não existe. O model pode não ter sido importado antes do `create_all()`, ou o banco está errado.
`duplicate key value violates unique constraint`	Alguém tentou inserir um valor único repetido. A validação da rota deveria ter capturado; se chegou ao banco, pode ser concorrência.
`violates foreign key constraint`	Tentativa de excluir algo que ainda é referenciado, ou inserir uma FK inexistente. Verifique a ordem das exclusões.
`Working outside of application context`	Uso de `db` fora de uma requisição ou de um `with app.app_context()`. Envolva a chamada no contexto.

ERRO	SIGNIFICADO E SOLUÇÃO
`RuntimeError: Working outside of request context`	Uso de `request` ou `session` fora de uma requisição, comum em scripts.
`BuildError: Could not build url for endpoint`	`url_for` com nome de função errado ou blueprint não registrado. Confira o nome qualificado.
`TemplateSyntaxError`	Erro no Jinja: bloco não fechado, variável mal formatada ou filtro inexistente. A mensagem indica linha e arquivo.
`ImportError: cannot import name ...`	Import circular ou arquivo renomeado. Lembre que models importam `config.db`, não o app.
Página recarrega sem mensagem	O `flash` foi chamado, mas o template não tem o bloco de mensagens, ou a sessão não está funcionando.
Formulário "não faz nada"	Campo com `name` diferente do esperado pela rota, ou JavaScript quebrado impedindo o submit. Verifique o console do navegador.

12.5 O arquivo sistema.py

`rotas/sistema.py` contém duas rotas que duplicam o que está em `app.py`: `/` e `/teste-banco`. Ele não é registrado, então não entra em conflito. Mantê-lo gera confusão: alguém pode editar a versão errada. A decisão recomendada é remover o arquivo ou passar a usá-lo de verdade, registrando o blueprint e retirando as rotas duplicadas do `app.py`. Se for registrado sem remover as rotas existentes, o Flask acusará conflito de endpoint.

12.6 Registro de logs

Todas as operações de escrita usam `current_app.logger.exception("...")` dentro do `except`. Isso grava a mensagem e o stack trace no log do Flask, enquanto o usuário recebe apenas uma mensagem amigável. Para investigar um erro, olhe o terminal onde o servidor está rodando. Em produção, configure um handler de arquivo ou um serviço de logs.

12.7 Dados de catálogo e importação

O catálogo tem milhares de registros que não foram digitados pela interface. Eles vieram de uma carga externa, e o campo `dados_originais` preserva o payload bruto. O script de importação não faz parte do repositório atual. Se for necessário repetir a carga, o caminho é criar um script idempotente que leia a fonte, valide fabricante/modelo e use `INSERT ... ON CONFLICT DO NOTHING` ou uma checagem prévia para não duplicar.

ANTES DE IMPORTAR DE NOVO

Faça backup. Uma carga malfeita pode criar milhares de duplicatas ou associar modelos ao fabricante errado. Valide uma amostra pequena antes de rodar o volume completo e prefira transações: ou tudo entra, ou nada entra.

13 Dívidas técnicas e próximos passos

Todo projeto tem escolhas que funcionam hoje e custariam caro amanhã. Esta seção lista as principais de forma honesta, com uma prioridade sugerida. Nenhuma delas impede o uso atual; todas melhoram a robustez, a segurança ou a manutenção.

13.1 O que melhorar, por prioridade

#	ITEM	PRIORIDADE	POR QUÊ
1	Migrações de banco (Alembic / Flask-Migrate)	**Alta**	Elimina o risco de schema divergente. O caso do `slug` mostrou que alterações manuais são fáceis de esquecer.
2	Segredos e senha fora do código	**Alta**	`SECRET_KEY`, credenciais do banco e senha de login estão versionados. Mover para ambiente e usar hash.
3	CSRF nos formulários	**Alta**	As operações de exclusão são POST, mas sem token. Flask-WTF resolve com pouco código.
4	`base.html` com blocos	**Média**	O cabeçalho e o rodapé estão duplicados em 15 arquivos. Centralizar reduz erros e facilita mudanças de navegação.
5	Unicidade composta de modelo por fabricante	**Média**	Hoje `modelo` é único globalmente. O correto é `(fabricante_id, modelo)`.
6	Constraints compostas para interfaces e portas	**Média**	A unicidade por modelo é validada só na aplicação e pode falhar em concorrência.
7	Otimizar a listagem de portas (N+1)	**Média**	Uma consulta por porta funciona, mas não escala. Uma consulta com `IN` resolve.
8	Testes automatizados	**Média**	Hoje a verificação é manual. Testes de rota com `test_client` e pytest protegeriam as validações.
9	Padronizar `conexoes.html` e `nova_conexao.html`	**Baixa**	São os dois templates que ainda usam o cabeçalho antigo, sem menu e sem "Sair".
10	Remover `rotas/sistema.py`	**Baixa**	Código morto que duplica rotas do `app.py` e confunde a manutenção.
11	Imports explícitos em vez de `from flask import *`	**Baixa**	Melhora legibilidade e ferramentas de análise sem alterar o comportamento.
12	Histórico e auditoria	**Baixa**	Timestamps e registro de quem alterou o quê. Só faz sentido depois de usuários reais.
13	Edição e desativação de portas do equipamento	**Baixa**	Hoje as portas nascem do modelo e não há tela para habilitar/desabilitar uma porta específica.

13.2 Roteiro sugerido

Curto prazo — deixar sólido

- Configurar Flask-Migrate e gerar a primeira migração do schema atual.
- Mover segredos para variáveis de ambiente e documentar o `.env.example`.
- Criar o `base.html` e migrar os templates aos poucos.
- Adicionar CSRF e testes de rota para os fluxos críticos.

Médio prazo — deixar completo

- Ajustar as constraints de unicidade para o domínio real.
- Otimizar consultas e adicionar índices onde houver filtro frequente.
- Criar uma tela de edição das portas do equipamento.
- Padronizar os dois templates legados e remover o código morto.

Longo prazo — deixar profissional

- Usuários, perfis e permissões (Flask-Login).
- Auditoria de alterações e histórico de conexões.
- Importação versionada do catálogo, com validação e relatório.
- API REST completa e, se necessário, uma interface separada.
- Implantação com Waitress/Gunicorn, HTTPS e backup automatizado.

> **UMA DÍVIDA NÃO É UM FRACASSO**
>
> Dívida técnica é uma decisão adiada, não um erro. O importante é que ela esteja registrada, compreendida e com um caminho de saída. Este capítulo existe para isso: quem assumir o projeto sabe exatamente onde estão os atalhos.

14 Glossário

TERMO	DEFINIÇÃO NO CONTEXTO DO NETLAB
App context	Contexto que informa ao SQLAlchemy qual aplicação e qual banco usar. Necessário para operar o `db` fora de uma requisição.
Blueprint	Agrupador de rotas de um mesmo assunto, registrado no app. Permite dividir o sistema em arquivos.
Cascade	Comportamento de propagar operações (como exclusão) do pai para os filhos. No catálogo, é automático; no inventário, é manual.
CRUD	Create, Read, Update, Delete: as quatro operações básicas sobre uma entidade.
Commit	Confirma a transação no banco. Antes dele, nada é definitivo.
Endpoint	Nome interno de uma rota, usado por `url_for` e por `before_request`.
FK (chave estrangeira)	Coluna que aponta para a chave primária de outra tabela, garantindo integridade referencial.
Flash	Mensagem temporária guardada na sessão e exibida uma vez na página seguinte.
Flush	Envia as alterações ao banco dentro da transação, sem confirmar. Usado para obter o `id` gerado.
Jinja2	Motor de templates que transforma HTML com marcações em HTML final, no servidor.
JSON	Formato de dados usado na API e no campo `dados_originais`.
Migrations	Versionamento controlado de alterações de schema. O projeto ainda não usa.
Model	Classe Python que descreve uma tabela. Não é o mesmo que CRUD.
ORM	Mapeamento objeto-relacional: classes e objetos em vez de SQL manual.
Paginação	Divisão de uma listagem em páginas. Usada em modelos (25) e interfaces (50).
Rollback	Desfaz a transação em caso de erro, devolvendo o banco ao estado anterior.
Sessão	Estado por usuário mantido em cookie assinado. Guarda apenas o nome do usuário autenticado.
Slug	Identificador amigável para URL. Foi removido do projeto; a coluna permaneceu no banco até ser dropada.
U (unidade de rack)	Medida de altura de equipamentos em rack. 1U equivale a 44,45 mm.
Uplink	Conexão entre equipamentos de rede, geralmente switch para switch ou switch para roteador.

A Apêndice A — Mapa de arquivos

ARQUIVO	CONTEÚDO
`app.py`	App, configuração, blueprints, login, logout, index, teste-banco, create_all.
`config.py`	Instância única do SQLAlchemy.
`models/regiao.py`	Região com auto-relacionamento.
`models/fabricante.py`	Fabricante com nome único.
`models/modelo_equipamento.py`	Modelo com dados técnicos e cascatas.
`models/interface_modelo.py`	Interface prevista no modelo.
`models/porta_alimentacao_modelo.py`	Entrada de energia do modelo.
`models/equipamento.py`	Instância de modelo em uma região.
`models/interface_equipamento.py`	Porta física do equipamento.
`models/conexao.py`	Cabo entre duas portas.
`rotas/regioes.py`	CRUD, hierarquia, filtro por subárvore, ciclos.
`rotas/fabricantes.py`	CRUD de fabricantes.
`rotas/modelos.py`	Modelos, interfaces, portas de alimentação, validação.
`rotas/equipamentos.py`	Equipamentos, cópia de portas, API, exclusão em cascata.
`rotas/conexoes.py`	Criação, assistente e desconexão.
`rotas/sistema.py`	Legado não registrado.
`templates/`	18 templates Jinja.
`static/style.css`	Tema Termix e layout responsivo.
`requerimentos.txt`	Dependências (UTF-16).
`GUIA_CRUD.md`	Guia de referência dos CRUDs.

B Apêndice B — Comandos úteis

Ambiente

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requerimentos.txt
python app.py
```

Banco

```
CREATE DATABASE netlab;

-- inspecionar contagens
SELECT 'regioes' AS tabela, COUNT(*) FROM regioes
UNION ALL SELECT 'fabricantes', COUNT(*) FROM fabricantes
UNION ALL SELECT 'modelos', COUNT(*) FROM modelos_equipamento;

-- ver a estrutura de uma tabela
\d+ modelos_equipamento
```

Verificação rápida de rotas

```
python -m py_compile app.py
python -m py_compile rotas\regioes.py rotas\fabricantes.py
python -m py_compile rotas\modelos.py rotas\equipamentos.py rotas\conexoes.py

git diff --check
```

Backup do banco

```
pg_dump -U postgres -d netlab -f backup_netlab.sql
```

Restauração

```
psql -U postgres -d netlab -f backup_netlab.sql
```

C Apêndice C — Checklist de verificação

Use esta lista depois de qualquer alteração estrutural. Todas as rotas abaixo devem responder **200** com a sessão autenticada (exceto as de escrita, que devem redirecionar **302**).

ROTA	ESPERADO	O QUE VALIDA
`/login`	200	Tela de login e CSS.
`/` sem sessão	302 → /login	Proteção global.

ROTA	ESPERADO	O QUE VALIDA
`/` autenticado	200	Página inicial e menu com "Sair".
`/regiao`	200	Árvore, filtro e formulário.
`/equipamentos`	200	Listagem, filtro e selects.
`/fabricantes`	200	CRUD de referência.
`/modelos`	200	Paginação e filtro por fabricante.
`/conexoes`	200	Listagem de cabos.
`/equipamentos/<id>`	200	Detalhes do equipamento.
`/equipamentos/<id>/portas`	200	Portas e estados.
`/equipamentos/<id>/editar`	200	Formulário de edição.
`/fabricantes/<id>/editar`	200	Edição de fabricante.
`/modelos/<id>/editar`	200	Edição de modelo.
`/modelos/<id>/interfaces`	200	Paginação de interfaces.
`/modelos/<id>/portas-alimentacao`	200	Portas de energia.
`/nova-conexao/<porta_id>`	200	Assistente de conexão.
`/logout`	302 → /login	Limpeza da sessão.
`/static/style.css`	200	CSS acessível sem login.

Nota sobre este documento

Este tutorial foi produzido a partir da leitura direta do código-fonte do projeto: `app.py`, `config.py`, os oito arquivos de `models/`, os seis arquivos de `rotas/`, os dezoito templates e o `static/style.css`. As contagens de tabelas refletem o banco `netlab` no momento da geração. As versões de bibliotecas foram lidas do ambiente virtual do projeto.

O objetivo foi explicar não só *o que* o NetLab faz, mas *por que* cada decisão foi tomada — incluindo as que têm limites conhecidos. Um sistema pequeno, legível e honesto sobre suas dívidas é mais útil do que um sistema grande que ninguém consegue manter.